Dedications

To my beloved parents,

With deepest gratitude, I dedicate this work to you. Your endless support, unwavering faith, and wise guidance have been the pillars of my academic journey. Your love and encouragement have inspired me to overcome challenges and strive for excellence. I am forever grateful for the strength you instilled in me, which made this achievement possible.

To my cherished siblings and dear friends,

Your constant motivation and companionship brightened every step of this path. Your belief in my potential provided me with the confidence to persist through difficulties. Thank you for standing by me with kindness and positivity, making this journey both joyful and meaningful.

To everyone who supported me,

Your friendship, care, and encouragement have left a lasting impact on my growth and success. I appreciate your guidance and the sincerity with which you shared your support. This accomplishment is as much yours as it is mine.

With heartfelt thanks,
Ahmed Cherif

Acknowledgments

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project.

First and foremost, I thank the professional supervisor, Mr. Mohamed Mhiri for his invaluable guidance, support, and encouragement during my internship. Their expertise and practical insights greatly shaped the development of this Restaurant management system.

My deepest appreciation extends to my academic supervisor, Mrs. Raoudha Nouisser, whose unwavering advice, accessibility, and encouragement provided essential academic direction throughout the project.

I am also grateful to the faculty members of North American Private University - International Institute of Technology for their continual support and for imparting the knowledge that formed the foundation of this work.

Finally, I thank the jury members, Mr. Taoufik Ben Abdallah (Chairman) and Mrs. Afef Latrach (Examiner), for dedicating their time to evaluate my work and deliver valuable feedback that helped improve the quality of this report.

Contents

List of Figures

List of Tables

Introduction

Recently, coffee shops and restaurants have played a key role in achieving business success and profitability. However, this business faces several challenges, such as handling fresh food that can spoil, ensuring orders are correct and delivered quickly, and understanding how their business is performing. Another crucial problem is food waste, which involves discarding food that could have been used. This issue costs a lot of money and also harms the environment. Studies show that food waste costs the food service industry billions of dollars every year (NRD22), and restaurants often lose a lot of their food stock because it goes bad, too much is prepared, or it's not used in the right order (FAO23). Even a small decrease in waste, like just 1%, can save a lot of money for restaurant owners over time (NRD22). Solving this problem is not just about improving how things work, it's necessary to save money and help the planet by wasting less food. Additionally, existing systems have different features (Hos22), however, their attention has not been payed to stop food waste

In our project, we seek a performant solution for managing small and medium restaurants, as well as cofee shops. Our main goal is to develop a robust system to effectively manage daily tasks and address certain problems, such as poor food stock management, mistakes in taking orders, and not having enough information to make smart choices (Smi22).

Overall, the proposed Restaurant Management Software is characterized by a simple, clear design tailored for non-technical staff, with local needs by supporting the French language and the Tunisian Dinar (TND). It enhances efficiency by improving food stock control, simplifying order handling, and providing data-driven insights, aiming to reduce waste, boost productivity, and support environmental sustainability for coffee shops and restaurants.

This report is composed of 3 chapters in addition to the introduction and conclusion of our project topic. We provide a brief description for each chapter:

- Chapter 1: General Context and Preliminary Analysis. This chapter introduces the Restaurant Management Software project, highlighting food wastage and inefficiencies in Tunisian restaurants, and proposes a user-friendly solution. It also outlines the system requirements and design, and defines the project workflow.

- Chapter 2: Conception and Analysis Phase. This chapter details the system design, including use case diagrams for modules like Menu Management and Loyalty Programs.

- Chapter 3: Realization and Implementation Phase. This chapter covers the system's development with Nuxt.js and Supabase, and testing to ensure functionality and usability.

Chapter 1

General Context and Preliminary Analysis

1.1 Introduction

This chapter outlines the foundational context for the Restaurant Management Software project and the preliminary analysis. It is divided into 4 main parts: the host organization, the project's scope and objectives, a detailed requirements analysis, and the adopted project methodology.

This chapter begins by defining the project's scope and the challenges faced by restaurants. Moreover, it identifies the requirement analysis, including actor identification, functional and non-functional requirements, the global use case diagram, and the class diagram. Finally, it presents the project methodology and the development workflow.

1.2 Presentation of the Host Organization

This section presents an overview of Sudotek, the host organization, detailing its background, mission, and key activities that support the development of the Restaurant Management Software.

1.2.1 Host Organization

SUDOTEK is an IT-focused company founded in 2021 by Mohamed Mhiri, serving as a one-stop solution for end-to-end software development.

Specializing in comprehensive software development, SUDOTEK supports all phases of the development lifecycle, from design and prototyping to development, implementation, testing, deployment, maintenance, and ongoing support. Our services include crafting intuitive web applications, robust server-side solutions, and seamless integrations using modern technologies.

With a customer-first approach, SUDOTEK is committed to understanding your unique goals and delivering scalable, high-performance solutions that drive success. Whether it's building a finance dashboard, a stock exchange platform, or a custom application, we prioritize quality, collaboration, and long-term support to bring your vision to life. Figure 1.1 presents the logo of SUDOTEK.

SUDOTEK

Figure 1.1: Logo of Sudotek

1.2.2 Company Activities

This company specializes in delivering comprehensive IT services across all phases of the software development lifecycle:

- **Design & Prototyping:** Wireframing, UI/UX design, and proof-of-concept development.
- **Development & Implementation:** Full-stack engineering (Frontend, Backend, DevOps).
- **Testing & QA:** Automated testing, security audits, and performance optimization.
- **Deployment & Integration:** Cloud hosting, CI/CD pipelines, and third-party API integration.
- **Maintenance & Support:** Proactive monitoring, updates, and scalability enhancements.

Furthermore, it adopts the customer-centric approach. Every project is treated as a partnership, with a focus on understanding client challenges, delivering customized solutions, and fostering transparency throughout the process. Its agility as a solo-led operation allows for rapid adaptation to evolving requirements while maintaining high standards of quality.

1.3 Project Scope

Sudotek aims to develop a comprehensive, web-based application that streamlines restaurant operations for small to medium-sized coffee shops and restaurants. The software is designed to be affordable, user-friendly, and efficient, focusing on reducing food wastage, optimizing workflows, and enabling data-driven decisions to improve profitability and sustainability (NRA24).

1.3.1 Problem Statement

The restaurant industry, particularly smaller enterprises, grapples with significant operational hurdles that impact profitability (Zha24). These challenges include:

- **Inefficient Inventory Management:** Manual tracking is error-prone, leading to overstocking or stockouts, as real-time visibility and usage tracking are often lacking.

- **Disorganized Order Processing:** Reliance on manual systems like handwritten tickets causes miscommunication, delays, and errors, increasing waste and affecting customer satisfaction.
- **Catastrophic Food Wastage:** Food waste leads to substantial financial losses, occurring through over-ordering due to poor forecasting, improper storage lacking FIFO principles, spoilage of perishables, over-preparation, and errors in orders. Studies show that restaurants waste 4–10% of purchased food before it reaches customers (BWaste25). Food waste in Tunisia is significant, with bread being the most wasted, at 113,000 tonnes yearly, or 15.7% of purchases (ResGate25)
- **Financial Impact of Wastage:** The US restaurant industry spends over $160 billion annually on waste-related costs (TheResto25), including food and packaging, with even a 1% reduction potentially saving thousands annually.
- **Challenges with Existing Restaurant Management Software Solutions:** Many tools are costly, designed for larger businesses, or lack features like French language support, TND handling, offline functionality, or detailed recipe-linked inventory control.

To sum up, this project presents a Restaurant Management Software that aims to address these issues by providing real-time visibility, streamlined processes, and tailored features specifically designed for small businesses in the Tunisian context.

1.3.2 Comparative Analysis

An in-depth analysis was conducted to shape the Restaurant Management Software's features, focusing on market trends, operational workflows, and user needs (Hos22). This analysis included:

- **Review of Existing Restaurant Management Software:** A study of available solutions like Restaurant365 (R36523), MarketMan (Mar23), and Plate IQ (Pla23) revealed common features such as inventory tracking and reporting, but identified deficiencies in recipe-based inventory management, French language support, and affordability for smaller establishments.
- **Observation of Restaurant Operations:** Local coffee shops and restaurants showed inefficiencies in manual inventory and order systems, leading to high error rates and time loss.
- **Stakeholder Interviews:** Restaurant managers and staff emphasized the need for simple interfaces, offline capabilities, and clear reporting.
- **Market Research:** Industry data underscored food wastage as a major issue, with significant potential for cost savings through better management practices.

Table 1.1 compares the proposed solution with existing software. This comparison highlights the opportunity for a tailored solution that better meets the needs of small restaurants in Tunisia, particularly in reducing food waste through advanced inventory and recipe management:

Table 1.1: Comparison of Restaurant Management Software Functionalities

Feature	**Restaurant365**	**MarketMan**	**Plate IQ**	**Proposed Solution**
Real-time inventory tracking	Yes	Yes	Yes	Yes
Recipe-based inventory management	Partial	Yes	No	Yes
Multilingual support (French)	No	No	No	Yes
Offline functionality	No	Partial	No	Yes
Waste logging and analysis	Partial	No	No	Yes
Local currency (TND) support	No	No	No	Yes
Affordable for small businesses	No	Partial	No	Yes
Kitchen workflow support	Yes	Yes	Partial	Yes

1.3.3 Proposed Solution

This project proposes a software for managing restaurants. Our proposed solution offers several features as well as new specific tasks. These features aim to reduce food wastage, enhance operational efficiency, and provide actionable insights, tailored for small to medium-sized restaurants in Tunisia:

- **Integrated Inventory Management:** Features include real-time stock tracking, automatic deductions based on recipes, low-stock alerts, FIFO enforcement, and waste analysis.
- **Recipe and Menu Management:** Enables precise recipe creation and links menu items to inventory.
- **Order Management:** Provides an intuitive interface for order entry, real-time tracking, and kitchen workflow support for admin and cook use.
- **Reporting and Analytics:** Offers insights into sales trends, wastage, and profitability to support strategic decisions.
- **User-Friendly Interface:** Designed with responsive layouts, role-based access, French language support, and offline capabilities.
- **Technical Features:** Includes web-based architecture, TND support, data security, and API integration for future scalability.

1.4 Requirement Analysis

This section elaborates on the requirement analysis phase for the Restaurant Management Software. It identifies the key actors, functional and non-functional requirements, global use case diagram, and class diagram.

1.4.1 Actors Identification

In the context of our project, we identified two primary actors that play critical roles in interacting with the Restaurant Management Software. Each actor addresses specific operational requirements:

- **Admin:** Restaurant owners or managers who oversee all system functionalities, including inventory tracking, menu configuration, order processing, and generating reports. Admins require comprehensive tools to make informed decisions and manage the restaurant effectively, addressing challenges like food wastage and inefficiencies.
- **Cook:** Kitchen staff who focus on order preparation, inventory updates related to cooking, and recipe management. Cooks need a simplified interface tailored to kitchen tasks, ensuring efficiency without access to sensitive administrative features.

1.4.2 Functional Requirements

This subsection presents the core functionalities that the Restaurant Management Software must deliver. They are organized by module to address key operational challenges such as inventory control and order processing. These requirements define the system's essential capabilities, prioritized to mitigate food wastage and enhance overall operational efficiency.

User Management:

- The system shall verify the user identity using email and password for secure access.
- Role-based access control shall restrict features based on user roles to ensure data security.

Inventory Management:

- Admins shall add inventory items, specifying name, category, unit, quantity, cost, and expiration date to track stock accurately.
- Admins shall view, update, or delete inventory items to maintain current records.
- Inventory tracking shall automatically deduct quantities based on completed orders to prevent stockouts.
- The system shall issue alerts for low stock levels based on customizable thresholds.
- Alerts shall notify users of approaching expiration dates for perishable items to reduce waste.
- Inventory reconciliation shall allow adjustments based on physical counts to ensure accuracy.
- A transaction history shall be maintained for auditing and tracking stock movements.
- Waste events shall be logged with quantities and reasons to analyze and minimize losses.

Recipe Management:

- Cooks shall create recipes, listing required inventory items and quantities to standardize preparation.

- Cooks shall view, modify, or delete recipes to adapt to menu changes.
- The system shall track recipe usage and popularity based on order data to inform menu planning.

Menu Management:

- Admins shall create menu items linked to recipes to streamline order processing.
- Admins shall view, update, or delete menu items to keep the menu current.
- Menu items shall have configurable prices to reflect costs and market conditions.
- Menu items and custom notes shall be categorized (e.g., appetizers, doneness, sauces) for clarity.
- Seasonal or time-based availability settings shall control menu item visibility.
- Admins shall add flexible custom notes to menu items to accommodate specific customer preferences.

Order Management:

- Admins shall create orders by selecting menu items and quantities for efficient processing.
- Order totals shall be calculated automatically based on menu item prices and quantities.
- Order statuses (Pending, In Preparation, Ready for Delivery) shall be tracked throughout fulfillment.
- Cooks shall update order statuses to reflect progress in the kitchen.
- Admins shall modify or cancel pending orders to handle changes.
- Inventory shall be automatically deducted based on recipes linked to ordered menu items.
- A complete order history shall be maintained for analysis and reporting.
- Basic offline order creation and status updates shall be supported during connectivity issues.

Customer Management:

- Admins shall create customers and assign orders to them.
- Historical statistics should be available in the reports page that provides the top customers based on the total spent.
- Admins shall assign orders to already existing customers by typing their number, and their profile displays in the create order page.
- Admins shall edit , delete, and search for existing customers in the reports page.

Reporting and Analytics:

- A dashboard shall display key performance indicators (KPIs) like revenue, order volume, and popular items.
- Sales reports shall summarize revenue and order counts by period (daily, weekly, monthly).
- Inventory usage reports shall highlight consumption patterns and potential waste areas.
- Menu performance reports shall show item popularity and profitability for strategic decisions.
- Waste tracking reports shall identify sources and patterns of inventory loss.
- Reports shall be exportable in CSV format for accessibility.
- Predictive analytics shall forecast inventory needs based on historical sales and stock levels.

These requirements were prioritized to address critical restaurant challenges, such as reducing food wastage through precise inventory and waste tracking, and improving efficiency via automated order and menu management.

1.4.3 Non-functional Requirements

This subsection presents the non-functional requirements of the Restaurant Management Software. They ensure that the software remains user-friendly, reliable, and secure while adhering to both user expectations and technical standards. These requirements contribute to the system's overall usability, maintainability, and robustness in real-world operational environments:

- **Usability:**
  - The interface shall be intuitive, requiring minimal training for restaurant staff with varying technical skills.
  - The system shall be responsive across devices, including desktops, laptops, and tablets, for flexible use.
  - Clear error messages and guidance shall assist users when input errors or failures occur.
  - Consistent navigation patterns shall ensure ease of use across all modules.
- **Performance:**
  - Initial pages shall load within 3 seconds under standard network conditions.
  - Order creation and updates shall process within 2 seconds to maintain workflow efficiency.

  - The system shall handle up to 10 concurrent users without performance degradation.
  - Standard reports (up to 1 month of data) shall generate within 5 seconds.

- **Reliability:**
  - The system shall achieve 99% uptime during restaurant operating hours.
  - Basic offline functionality shall support critical operations during internet disruptions.

- **Security:**
  - Sensitive data, including user credentials and financial details, shall be encrypted.
  - Strong password policies shall enhance account security.
  - Role-based access control shall limit feature access by user role.
  - Significant actions shall be logged for audit purposes.

- **Maintainability:**
  - A modular architecture shall simplify future enhancements.
  - Comprehensive user and developer documentation shall support system maintenance.
  - Consistent coding standards shall be followed across the codebase.
  - Automated tests shall cover at least 70% of the codebase for reliability.
  - Detailed error logging shall facilitate troubleshooting.

1.4.4 Global Use Case Diagram

Figure 1.2 presents the global use case diagram. This diagram outlines the interactions between Admins, Cooks, and the system's core features, providing a high-level view of functionality:

Core Use Cases

We present a brief description for each use case shown in Figure 1.3. More details will be explored in the next chapter.

- **Manage Inventory, Menu, Orders, Customers, and Loyalty Settings (Admin):** These use cases allow the Admin to oversee and control the critical business components. Each of these actions includes authentication to ensure that only authorized users can perform sensitive operations.

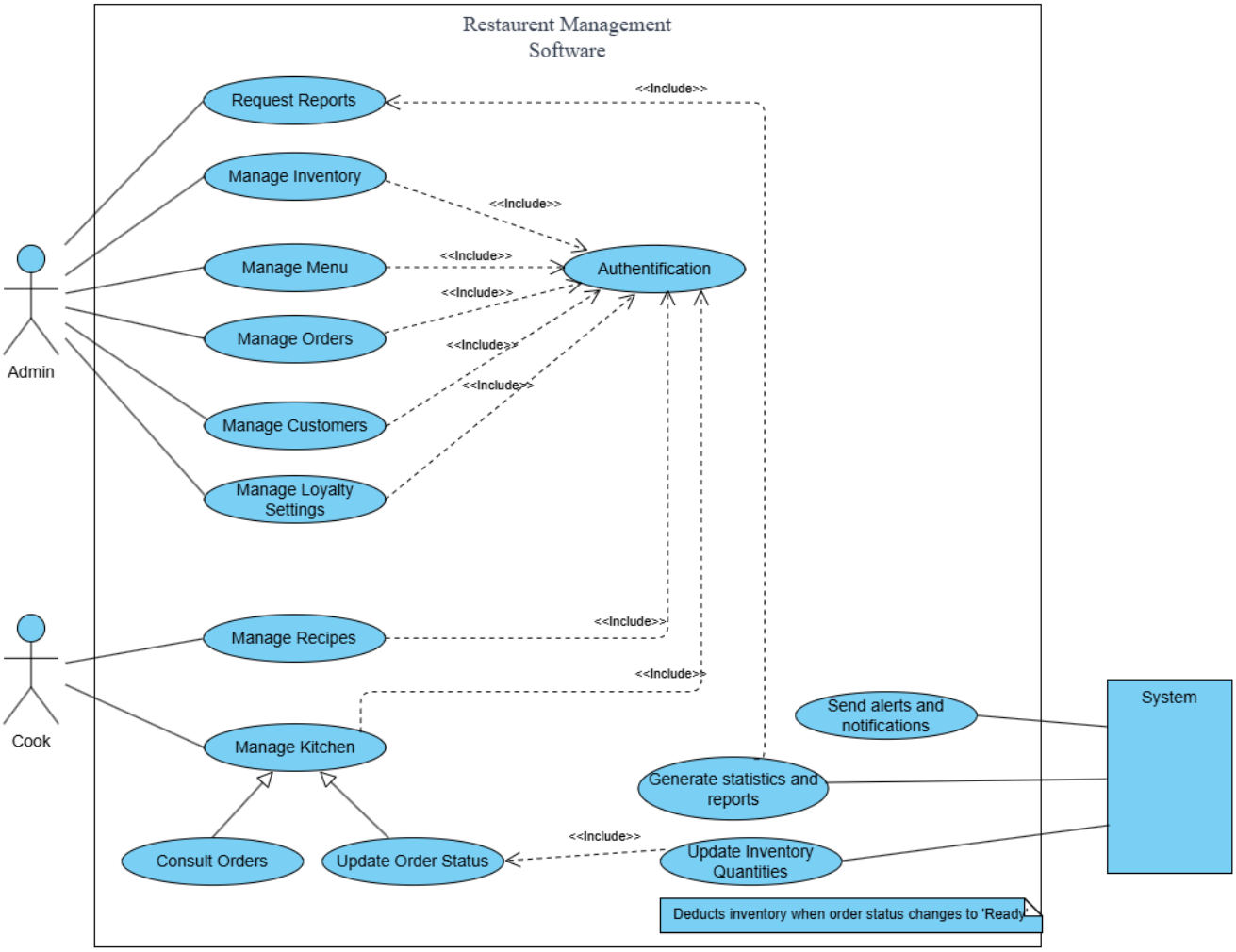

Figure 1.2: Global Use Case Diagram

- **Manage Recipes and Kitchen (Cook):** The Cook uses these features to prepare dishes based on recipes and to organize the workflow within the kitchen. These use cases also require prior authentication.
- **Authentication:** A shared use case included in all primary operations to enforce secure access and role-based permissions.
- **Generate Statistics and Reports:** This feature allows users (primarily Admins) to analyze business performance through automated reporting tools. It is included in the kitchen management process for operational insights.
- **Update Inventory Quantities:** This automated use case ensures that inventory levels are accurately updated when order statuses change (e.g., when an order is marked as "Ready"). It helps maintain real-time stock levels.
- **Send Alerts and Notifications:** The system automatically triggers alerts or notifications based on predefined events, such as low inventory levels or completed orders.
- **Note (Automated Inventory Deduction):** A note is included in the diagram to specify that inventory is automatically deducted by the system when the order status changes to "Ready", minimizing manual stock management errors.

1.4.5 Class Diagram

Figure 1.3 presents the class diagram. It defines the system's data structure, informing the database schema and application logic.

Class Diagram Description

The class diagram, illustrated in Figure 1.3, provides a high-level object-oriented representation of the Restaurant Management Software. It captures the system's core entities, their attributes, relationships, and responsibilities.

User is the base class that generalizes both **Admin** and **Cook** roles. Each user has attributes such as `first_name`, `last_name`, `email`, `password_hash`, `phone`, `hire_date`, and `role`.

Admin inherits from **User** and is responsible for managing several key components of the system:

- **Customer**: Includes information such as name, contact, address, loyalty points, and total spending.
- **Order**: Represents customer orders with fields like `total`, `created_at`, `status`, `discount_amount`, and `discount_reason`.
- **Menu_items**: Represents the dishes offered in the restaurant with attributes including `name`, `description`, `price`, and `preparation_time`. Each item belongs to a **Menu_category**.
- **Inventory_items**: Tracks kitchen stock with fields such as `name`, `description`, `quantity_in_stock`, `reorder_level`, `unit_cost`, and `has_expiration`. Each item is categorized under an **Inventory_category**.

Cook also inherits from **User** and is responsible for updating order status through the kitchen interface and managing the **Recipes**. Each recipe includes a `name`, `description`, and a list of preparation `steps`.

The system incorporates multiple composition relationships:

- **Menu_items** are associated with one **Menu_category**.
- **Inventory_items** belong to one **Inventory_category**.

This class diagram helps define the core business logic and data relationships for implementing robust and maintainable restaurant management software.

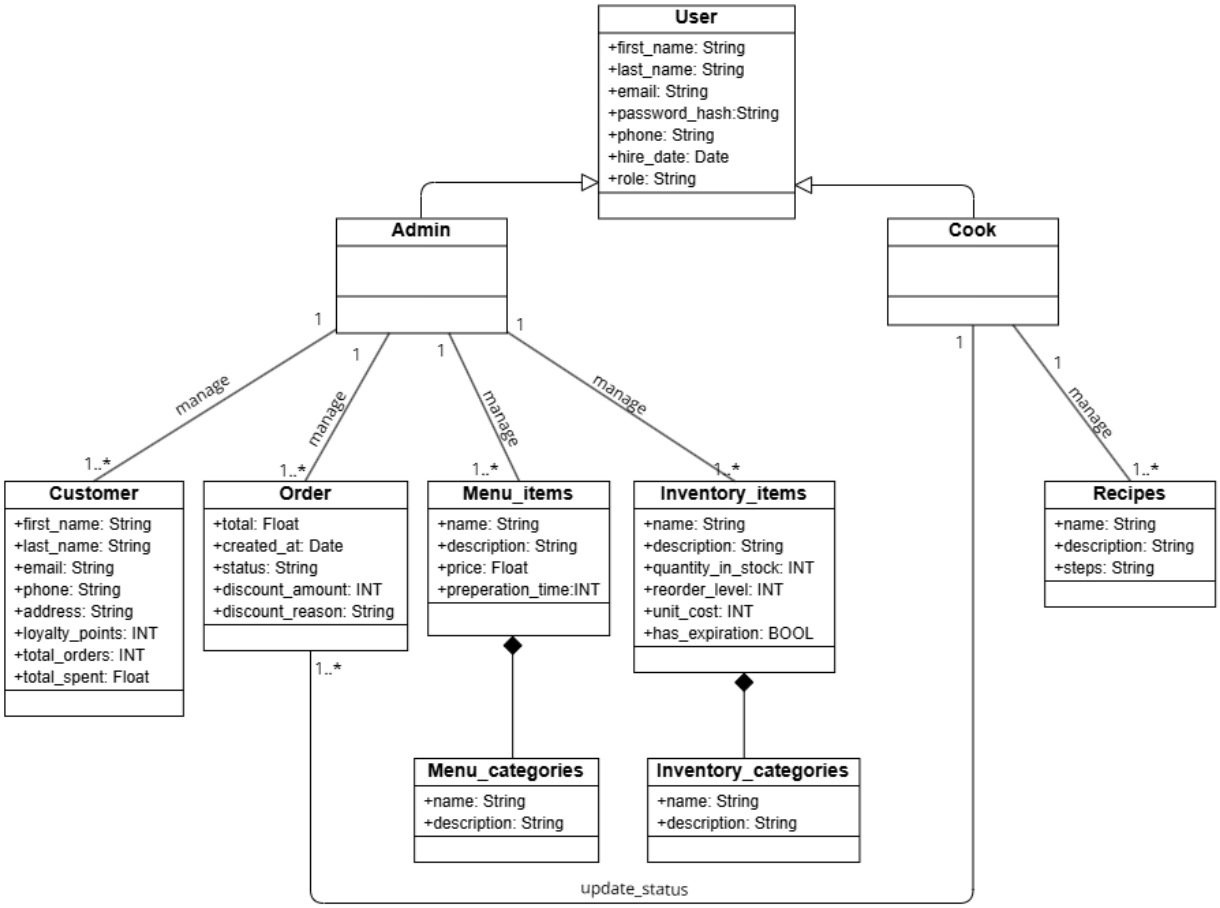

Figure 1.3: Class Diagram

1.5 Project Methodology

This section presents the project management approach used for developing the Restaurant Management Software, focusing on the Kanban methodology and its implementation.

1.5.1 Kanban Methodology

The Kanban methodology (And22), adapted from Toyota's manufacturing system, was chosen for its alignment with the project's needs. Its benefits include:

- **Flexibility and Adaptability:** Kanban allowed the team to adjust priorities as requirements evolved, which was crucial given the project's dynamic nature.
- **Workflow Visualization:** A visual board helped track progress, identify bottlenecks, and maintain transparency across the team.
- **Continuous Delivery Focus:** By limiting work in progress (WIP), Kanban ensured steady feature delivery and early feedback.
- **Low Overhead:** Its simplicity minimized administrative burden, fitting the small team's constraints.
- **Evolutionary Improvement:** Incremental process adjustments improved efficiency over time.

A Kanban board was set up with stages (Backlog, Ready, In Progress, Testing, Done), using cards to represent tasks and WIP limits to maintain flow, as illustrated in Figure 1.4. Daily stand-ups facilitated progress reviews and priority adjustments, ensuring adaptability and efficiency (Wil23).

Figure 1.4: Kanban Process Illustration

The Kanban implementation ensures effective progress while adapting to the project's evolving needs. The Kanban implementation includes:

- **Workflow Definition:** Stages were defined as Backlog, Ready, In Progress, Testing, and Done, ensuring clear task progression.
- **Work Item Types:** Included user stories, technical tasks, bugs, and documentation, covering all project aspects.
- **Prioritization:** Tasks were prioritized based on business value, dependencies, complexity, and stakeholder feedback, focusing on food wastage reduction.
- **WIP Limits:** Limits (e.g., 2 items per developer in Progress, 3 in Testing) prevented overloading and ensured steady progress.
- **Metrics and Monitoring:** Metrics like lead time, cycle time, throughput, and cumulative flow tracked performance.
- **Continuous Improvement:** Retrospectives drove process refinements, enhancing efficiency.

1.5.2 Product Backlog

Based on the previous requirement analysis, Table 1.2 outlines the product backlog of our project. It presents the project features and their priorities to address food wastage and efficiency, serving as the central requirements repository:

Table 1.2: Product Backlog

ID	Feature	Description	Priority
1	User Authentication	Enable user registration, login, and role-based access control	High
2	Inventory Management	Support CRUD operations for inventory with real-time tracking	High
3	Low Stock Alerts	Generate alerts for low inventory levels	Medium
4	Expiration Tracking	Monitor and alert for expiring inventory items	Medium
5	Recipe Management	Enable CRUD operations for recipes with ingredient details	High
6	Menu Management	Support CRUD operations for menu items linked to recipes	High
7	Order Creation	Allow order creation with menu item selection	High
8	Order Status Tracking	Enable status updates for orders during fulfillment	Medium
9	Inventory Deduction	Automatically update inventory based on orders	High
10	Basic Reporting	Provide sales and inventory reports	Medium
11	Advanced Analytics	Offer predictive inventory analytics	Low
12	Offline Mode	Support critical operations offline	Medium
13	Waste Logging	Enable logging and analysis of waste events	Medium
14	Multi-language Support	Add English and Arabic language support	Very Low
15	Data Export	Support report exports in EXCEL and CSV	Low

1.5.3 Kanban Board Planning

Table 1.3 visualizes our development process using the Kanban methodology ensuring transparency and efficiency. This process is elaborated via Trello, , which will be discussed in the next section.

Table 1.3: Kanban Board Planning

Column	**Purpose**
Backlog	Stores all identified tasks awaiting prioritization
Ready	Holds prioritized tasks ready for development with full specifications
In Progress	Tracks tasks being actively developed (WIP limit: 2 per developer)
Testing	Manages completed tasks under testing (WIP limit: 3)
Done	Contains fully completed and accepted tasks
Blocked	Lists tasks stalled by external dependencies

1.6 Utilizing Trello for Task Management

This section presents the use of Trello as a task management tool, detailing its implementation and benefits in the project.

1.6.1 Task Management with Trello

Trello was selected for its alignment with Kanban principles, providing a visual and intuitive platform for task management (Atl23). Its implementation involved:

- **Board Setup:** A Trello board was created with lists for Backlog, Ready, In Progress, Testing, Done, Ideas, and Blocked, reflecting the project workflow.
- **Card Structure:** Cards included titles, descriptions, acceptance criteria, labels, checklists, due dates, attachments, and comments (Tre22b).
- **Visual Indicators:** Labels, images, progress bars, and member assignments enhanced visibility.
- **Automation:** Trello's Butler automated tasks like moving cards, adding checklists, and notifying team members.
- **Integration:** Trello integrated with GitHub, Google Drive, and Discord, streamlining development workflows.

Trello's setup provided a centralized platform for tracking progress, minimizing training needs, and enhancing team focus on development. Figure 1.5 shows how we manage our tasks in Trello.

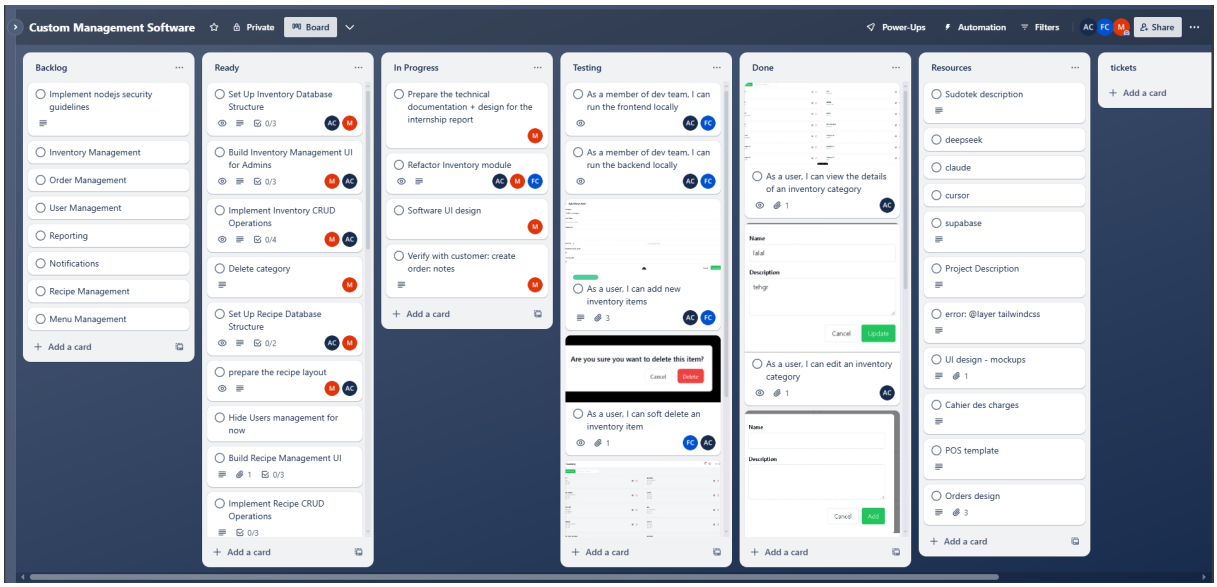

Figure 1.5: Task Management in Trello

1.6.2 Benefits of Using Trello

This subsection highlights the advantages of using Trello for the project, emphasizing its impact on efficiency and collaboration. Trello offered several benefits (Tech23; Collab23), including:

- **Enhanced Visibility:** Its visual board provided a clear overview of project status, aiding in bottleneck identification.
- **Improved Collaboration:** Features like comments and mentions fostered team communication and knowledge sharing .
- **Flexibility:** Trello's interface allowed quick workflow adjustments as the project evolved.
- **Low Overhead:** Minimal setup and intuitive design reduced administrative efforts.
- **Accessibility:** Cloud-based access and a mobile app ensured availability across locations.
- **Traceability:** Trello's history of changes supported retrospectives and knowledge transfer.
- **Integration:** Seamless links with GitHub and other tools connected task management to development.

Trello's features significantly contributed to the project's success, ensuring efficient task management and effective collaboration.

1.7 Conclusion

This chapter introduces the context for the Restaurant Management Software project, detailing Sudotek's role as a technology leader in Tunisia and addressing key restaurant challenges, such as food wastage, through features like inventory management and analytics. Besides, it explained the requirements analysis for the Restaurant Management Software, covering actors, functional and non-functional requirements. Moreover, it presented the utilized Kanban methodology, implemented via Trello, and provided a flexible framework for development, adapting to the project's needs.

Next chapter includes a detailed explanation of the conception phase related to our project. This stage serves for further implementation and development of the application.

Chapter 2

Conception and Analysis Phase

2.1 Introduction

The conception and analysis phase focused on understanding the business requirements and translating them into technical specifications that would effectively address the inefficient inventory management, disorganized order processing, and food wastage problem identified in Chapter 1. By thoroughly analyzing each feature before implementation, we ensured that the system would be cohesive, efficient, and aligned with the project's main goals of enhancing restaurants' workflow, optimizing sales, and reducing food wastage through improved inventory management and operational efficiency.

This chapter details the conception and analysis phase of the Restaurant Management Software, covering all features of the project. More specifically it presents the detailed use case diagrams for each module, sequence diagrams for key operations, and textual descriptions of critical use cases. This phase is crucial as it establishes the foundation for the implementation by defining the system's requirements, use cases, and interactions.

2.2 Overview of System Modules

This section provides a general overview of the key modules within the Restaurant Management Software, laying the groundwork for the detailed analysis that follows. Each module is designed to address specific operational needs while contributing to the overarching goal of minimizing food wastage. These modules are:

- **The Inventory Management** module serves as the backbone for tracking stock levels, managing expiration dates, and handling inventory transactions. It enables users to monitor quantities, categorize items, and manage batches with expiration tracking, utilizing a First-In-First-Out (FIFO) approach to reduce spoilage. This module also includes features for generating alerts on low stock or nearing expiration dates, ensuring timely restocking and waste prevention.
- **The Recipe Management** module facilitates the creation and maintenance of recipes by linking them to inventory items with precise quantities. It supports the definition of preparation steps, providing a tool to optimize ingredient usage.
- **The Menu Management** module allows users to design and update the restaurant's menu by associating menu items with recipes. It includes functionalities for

setting prices, and categorizing items, enabling efficient menu management that reflects current inventory availability.

- **The Order Management** module streamlines the process of creating, modifying, and tracking orders. It ensures accurate inventory deduction based on recipe-linked menu items, using FIFO logic for items with expiration dates, and supports offline order storage with synchronization when connectivity is restored. Additionally, it allows assigning customers to orders and applies loyalty discounts based on order history.
- **The Kitchen Management** module offers a specialized view for kitchen staff to manage order preparation and fulfillment. It allows cooks to update order statuses, mark items as ready, and record discarded items with reasons, integrating with wastage tracking to analyze and reduce food loss during preparation.
- **The Customers Management** module permits the process of creating, modifying, and tracking customers. It mainly simplifies the order delivery process by providing clear customer details, while it also helps in analyzing the business and giving statistics based on customers.
- **The Loyalty Settings Management** module authorizes the admin to create, edit and delete loyalty plans. A strong customer-restaurant relationship is the foundation, while a loyalty plan enhances it by making the customer feel recognized and rewarded for their continued support.
- **The Reporting module** provides analytical tools for Admins to generate insights into sales, inventory usage, and menu performance. It also manages customer information and loyalty programs, offering features to view, update, or delete customer records (including a default top 10 list and search functionality) and create, edit, or delete loyalty program rules, enabling data-driven decisions to optimize operations and reduce food wastage.

2.3 Detailed Use Cases

This section explains the use case diagram for each module in our project

2.3.1 Inventory Management Module

The Inventory Management module is designed to address the core goal of reducing food wastage by providing tools to track stock levels, manage expiration dates, and record inventory transactions.

As depicted in Figure 2.1, the Inventory Management use case diagram shows that Admin can perform all inventory-related actions, including adding, viewing, updating, and deleting inventory items, as well as managing categories and tracking stock levels.

Table 2.1 details the Update Inventory Item use case, which allows admins to update the details of an existing inventory item. This use case is particularly important as it handles not only basic updates but also specialized operations like batch management for items with expiration dates (using FIFO logic for deduction) and alert generation for low stock levels.

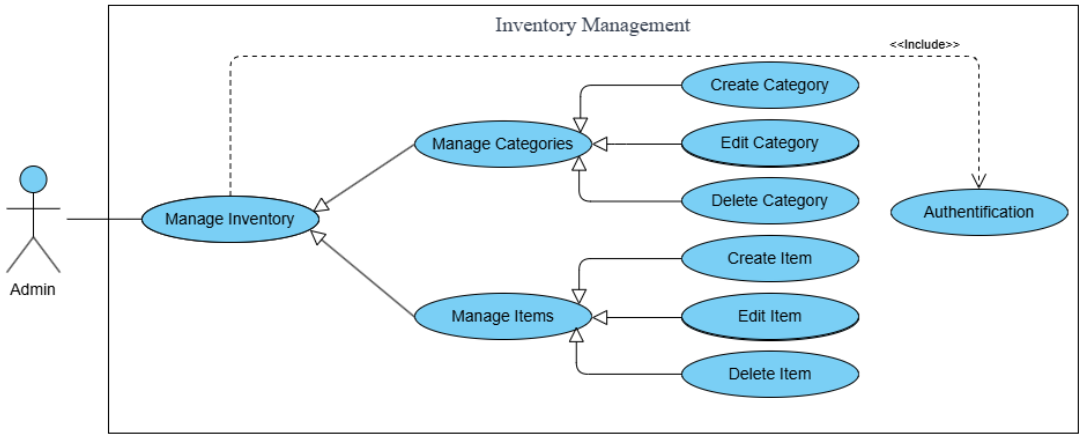

Figure 2.1: Use Case Diagram for Inventory Management

Table 2.1: Textual Description of Update Inventory Item

Use Case Name	Update Inventory Item
Actors	Admin
Description	This use case allows users to update the details of an existing inventory item.
Preconditions	1. The user is authenticated and logged into the system. 2. The inventory item exists in the system.
Basic Flow	1. User navigates to the Inventory Management section. 2. User locates the desired inventory item in the list. 3. User selects "Edit" for the item. 4. System displays the item details in an editable form. 5. User modifies the desired fields (name, category, unit, quantity, cost, expiration date, batches, details of each batch). 6. User submits the form. 7. System validates the input. 8. System updates the inventory item with the new details. 9. System displays a success message.
Alternative Flows	7a. If validation fails: 1. System displays error messages. 2. User corrects the input and resubmits. 8a. If updating quantity: 1. The system logs the quantity change in the transaction history. 2. If the new quantity is below the threshold, the system generates a low stock alert. 8b. If updating item with expiration tracking: 1. System updates or creates inventory batches with expiration dates using FIFO logic for deduction.

Use Case Name	Update Inventory Item
	2. System checks for approaching expiration dates and generates alerts if needed.
Postconditions	1. The inventory item is updated with the new details. 2. If applicable, a transaction record is created for quantity changes. 3. If applicable, low stock or expiration alerts are generated. 4. If applicable, inventory batches are updated.

2.3.2 Recipe Management Module

The Recipe Management module enables the creation and management of recipes, which are essential for tracking ingredient usage.

As shown in Figure 2.2, the Recipe Management use case diagram illustrates that Cooks can create, view, update, and delete recipes.

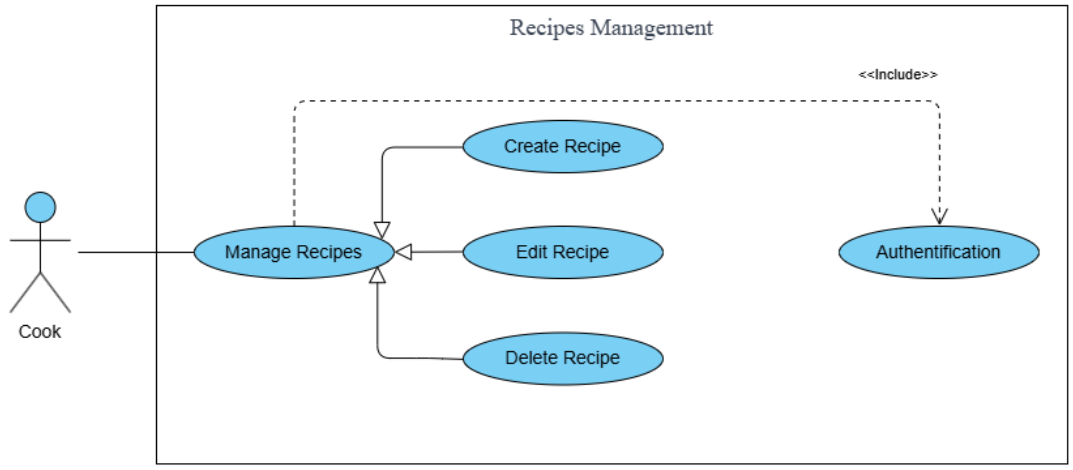

Figure 2.2: Use Case Diagram for Recipe Management

Table 2.2 provides a detailed description of the Create Recipe use case, which is fundamental to the recipe management module. This use case establishes the link between inventory items and recipes, which is essential for tracking ingredient usage.

Table 2.2: Textual Description of Create Recipe

Use Case Name	Create Recipe
Actors	Cook
Description	This use case allows users to create a new recipe with specified ingredients and quantities.
Preconditions	1. The user is authenticated and logged into the system. 2. Inventory items exist in the system to be used as ingredients.
Basic Flow	1. The user navigates to the Recipe Management section. 2. User selects "Create New Recipe".

Use Case Name	Create Recipe
	3. User enters recipe details (name, description, preparation time). 4. The user adds ingredients from the inventory, specifying quantities and units. 5. The user adds preparation steps. 6. The user submits the form. 7. The system validates the input. 8. The system creates the new recipe. 9. The system displays a success message.
Alternative Flows	8a. If validation fails: 1. The system displays error messages. 2. The user corrects the input and resubmits.
Postconditions	1. A new recipe is created in the system. 2. The recipe is available for use in menu items.

2.3.3 Menu Management Module

The Menu Management module allows users to create and manage menu items, linking them to recipes for order.

As illustrated in Figure 2.3, the Menu Management use case diagram shows that Admins can create, view, update, and delete menu items. The diagram also includes specialized use case "Manage notes", well organized custom notes where each menu item could have its own notes (e.g. Medium rare, with/without sauce, Extra spicy).

Table 2.3 details the Create Menu Item use case, which establishes the link between recipes and menu items as well as inventory items and menu items. This use case is important for the business aspects of restaurant management, as it involves setting prices.

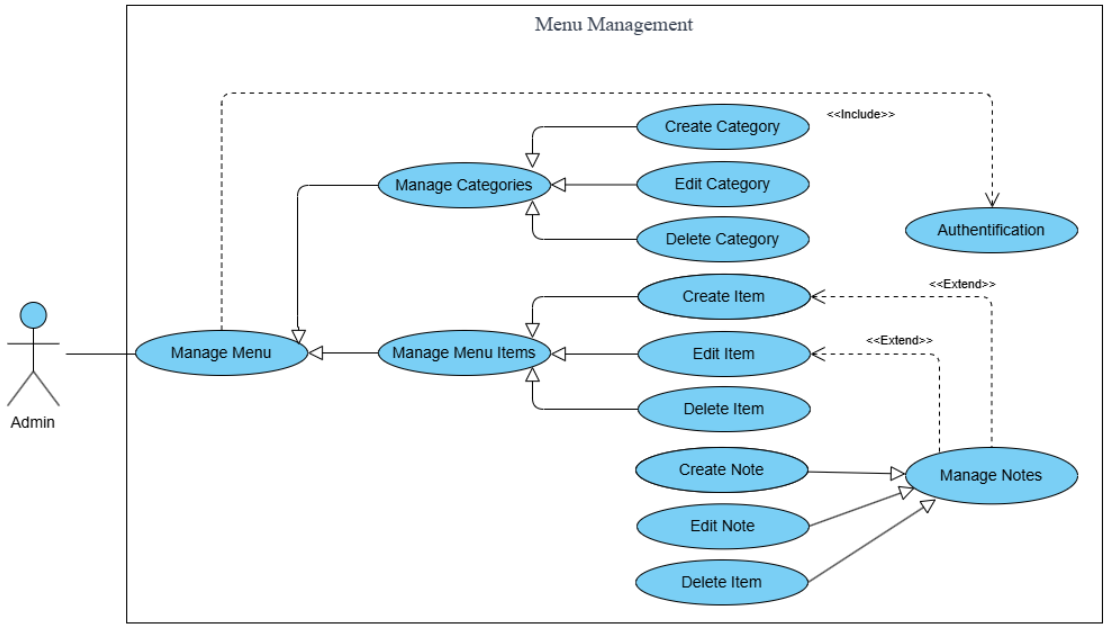

Figure 2.3: Use Case Diagram for Menu Management

Table 2.3: Textual Description of Create Menu Item

Use Case Name	Create Menu Item
Actors	Admin
Description	This use case allows users to create a new menu item linked to a recipe or inventory item.
Preconditions	1. The user is authenticated and logged into the system. 2. At least one recipe exists in the system.
Basic Flow	1. User navigates to the Menu Management section. 2. User selects "Create New Menu Item". 3. User enters menu item details (name, category, description, price, picture). 4. The user selects a source (inventory item/recipe). 5. The user selects a source item from the inventory or recipes, depending on the option selected in the source. 6. The user creates custom Notes' groups. 7. The user creates custom Notes' options under each group. 8. The user submits the form. 9. The system validates the input. 10. The system creates the new menu item. 11. The system displays a success message.
Alternative Flows	9a. If validation fails: 1. The system displays error messages. 2. The user corrects the input and resubmits.
Postconditions	1. A new menu item is created in the system. 2. The menu item is available for inclusion in orders.

2.3.4 Order Management Module

The Order Management module facilitates the creation and tracking of orders, ensuring accurate inventory deduction based on the recipes/inventory-items associated with ordered menu items.

As shown in Figure 2.4, the Order Management use case diagram illustrates that Admin can create, modify, and cancel orders, while Cook can view orders and update their status (More details are provided in the next subsection). The diagram also shows the relationship between order creation and inventory deduction, highlighting the system's integrated approach to tracking ingredient usage, along with customer assignment and loyalty discount application.

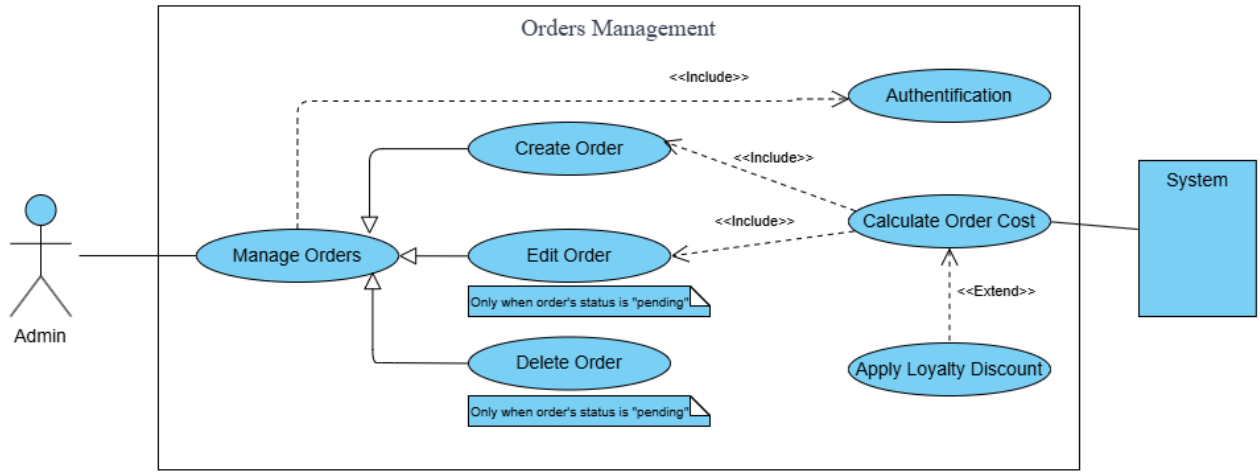

Figure 2.4: Use Case Diagram for Order Management

Table 2.4 provides a detailed description of the Create Order use case, which is a core operation in the order management module. This use case is particularly important as it triggers the inventory deduction process using FIFO logic for items with expiration dates, assigns a customer to the order, and applies loyalty discounts based on the customer's order history.

Table 2.4: Textual Description of Create Order

Use Case Name	Create Order
Actors	Admin
Description	This use case allows an Admin to create a new order in the system, including customer assignment and loyalty discount application.
Preconditions	1. The Admin is authenticated and logged into the system. 2. Menu items exist in the system.
Basic Flow	1. Admin navigates to the Order Management section. 2. Admin selects "Create New Order". 3. Admin selects menu items and specifies quantities. 4. Admin notes notes for order item. 5. The system calculates the order total. 6. Admin enters a customer phone number to assign to the order. 7. If the phone number exists, the system auto-assigns the existing customer; if not, the system prompts a form to enter new customer details (name, email, phone). 8. System updates the order cost in case a discount is applied. 9. Admin confirms the order. 10. The system verifies inventory availability for all required ingredients. 11. System creates the order with "Pending" status. 12. The system displays a success message with the order details.

Use Case Name	Create Order
Alternative Flows	6a. If the customer phone number is not found and the form is skipped: 1. The system proceeds without assigning a customer. 7a. If no loyalty discount applies: 1. The system proceeds without applying a discount. 10a. If offline: 1. The system stores the order locally. 2. The system synchronizes with the server when connectivity is restored.
Postconditions	1. A new order is created in the system with "Pending" status. 2. The order appears in the list of active orders for kitchen staff. 3. The customer is assigned to the order, and any applicable loyalty discount is applied.

2.3.5 Kitchen Management Module

The Kitchen Management module provides a streamlined interface for kitchen staff to manage orders, ensuring efficient preparation and fulfillment.

As illustrated in Figure 2.5, the Kitchen Management use case diagram shows that Cooks can view pending orders, update order status, and manage order items. The kitchen management functionality help the cook change status from "pending" to "In progress" or "ready". After updating the order status, the system modifies the inventory quantity.

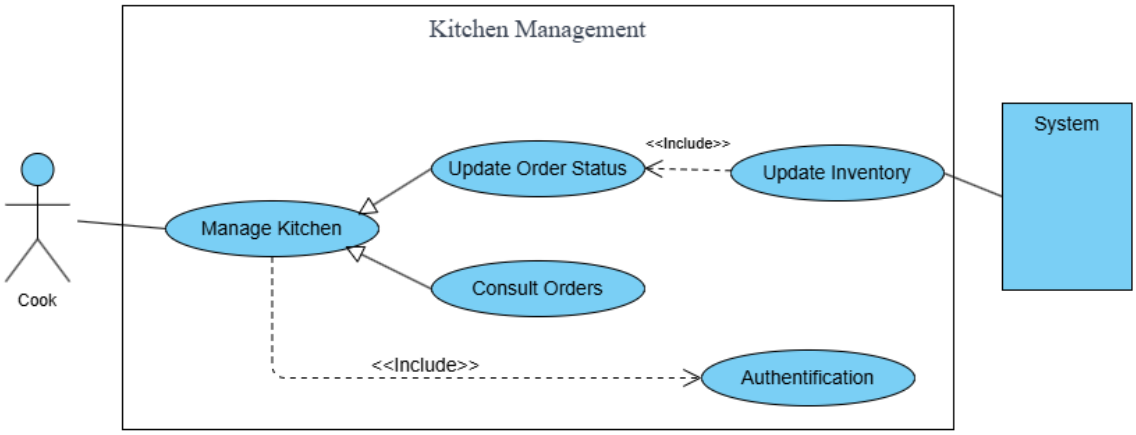

Figure 2.5: Use Case Diagram for Kitchen Interface

Table 2.5 details the Update Order Status use case, which is a core operation in the kitchen interface module. This use case is important for tracking the progress of orders through the kitchen and ensuring timely fulfillment, with the "Discard Item" flow recording wastage details for analysis.

Table 2.5: Textual Description of Update Order Status

Use Case Name	Update Order Status
Actors	Cook
Description	This use case allows a Cook to update the status of an order in the kitchen interface.
Preconditions	1. The Cook is authenticated and logged into the system. 2. The order exists in the system with "Pending" or "In Progress" status.
Basic Flow	1. Cook views the list of active orders in the kitchen interface. 2. Cook selects an order to update. 3. Cook changes the order status (e.g., from "Pending" to "In Progress" or from "In Progress" to "Ready"). 4. The system updates the order status in the database. 5. The system records the status change in the order history. 6. The system displays a success message.
Alternative Flows	3a. If discarding an item: 1. Cook selects the item to discard. 2. The system records the discarded item, its quantity, cost, and reason in the wastage log. 3. System updates the order to reflect the discarded item.
Postconditions	1. The order status is updated in the system. 2. The status change is recorded in the order history. 3. If applicable, discarded items are recorded for reporting.

2.3.6 Manage Customers Module

The Manage Customers module enables users to view, update, and remove customer profiles, tracking their spending to enhance customer relationship management.

As illustrated in Figure 2.6, the Manage Customers use case diagram shows that Admins can view, create, update, and delete customer data.

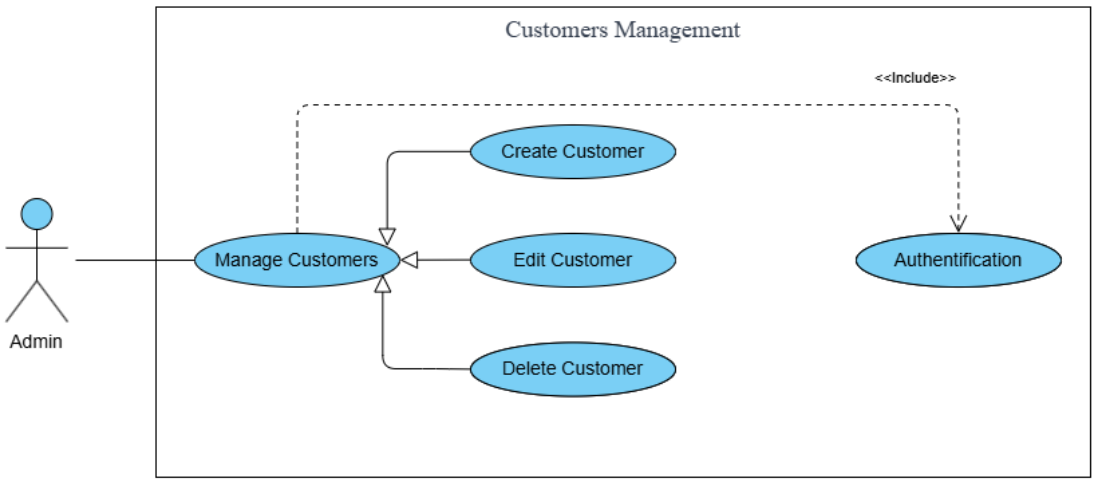

Figure 2.6: Use Case Diagram for Manage Customers

Table 2.6 details the Edit Customer Profile use case, which allows admins to update a customer's information.

Table 2.6: Textual Description of Manage Customers

Use Case Name	Manage Customers
Actors	Admin
Description	This use case allows users to edit or delete a customer's profile.
Preconditions	1. The user is authenticated and logged into the system. 2. At least one customer exists in the system with an order history.
Basic Flow	1. Admin navigates to the Reports page. 2. Admin navigates to the Manage Customers section. 3. Admin searches for a customer by phone number or name. 4. The system retrieves the customer's data. 5. Admin edits the customer's information. 6. Admin submits the customer data. 7. The system validates the input and saves the updated customer. 8. The system displays a success message. 9. Admin can delete an existing customer if needed.
Alternative Flows	4a. If customer not found: 1. The system displays a "Customer not found" message. 7a. If validation fails: 1. The system displays error messages. 2. Admin corrects the input and resubmits.
Postconditions	1. The customer is updated or deleted.

2.3.7 Manage Loyalty Settings Module

The Manage Loyalty Programs module allows users to create, update, and manage loyalty rules, such as discounts based on order frequency, to encourage customer retention.

As illustrated in Figure 2.7, the Manage Loyalty Programs use case diagram shows that Admins can create, view, update, and delete loyalty rules (e.g., 50% discount on the 5th order).

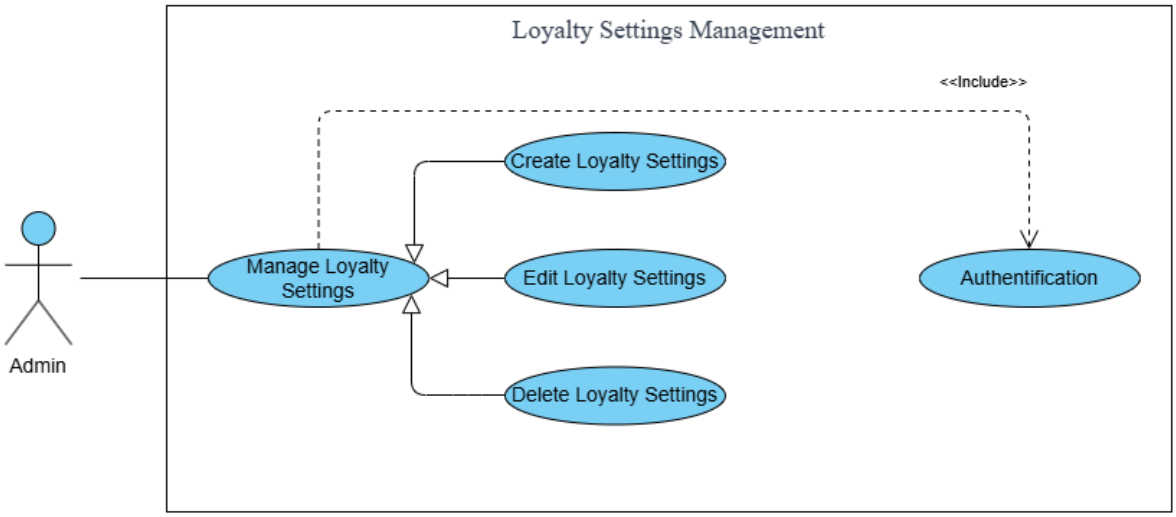

Figure 2.7: Use Case Diagram for Manage Loyalty Programs

Table 2.7 details the Set Loyalty Rules use case, which enables admins to define criteria for loyalty rewards, such as discounts based on order count.

Table 2.7: Textual Description of Manage Loyalty Programs

Use Case Name	Manage Loyalty Programs
Actors	Admin
Description	This use case allows an Admin to create, edit, or delete loyalty program rules, such as offering a 10% discount on a customer's 5th order.
Preconditions	1. The Admin is authenticated and logged into the system.
Basic Flow	1. Admin navigates to the Reporting section. 2. Admin selects the customer management interface. 3. Admin creates or edits a loyalty rule by specifying the order threshold (e.g., 5th order) and discount percentage (e.g., 50%). 4. Admin submits the rule. 5. The system validates the input and saves the new or updated rule. 6. The system displays a success message. 7. Admin can delete an existing rule if needed.
Alternative Flows	5a. If validation fails: 1. The system displays error messages. 2. Admin corrects the input and resubmits.
Postconditions	1. The loyalty program rule is created, updated, or deleted. 2. The rule is available for application during order creation.

2.3.8 Reporting and Alerts Module

The Reporting module provides analytical tools to monitor restaurant operations, with a focus on identifying opportunities to reduce food wastage.

This subsection presents the use case diagram for the Reporting and Sending Alerts module, outlining the reporting functionalities available to Admins.

As shown in Figure 2.8, the Reporting use case diagram illustrates that Admin have access to request a report in the other hand the system is responsible for that, as well as sending alerts in case there is low stock inventory item or near expiring one.

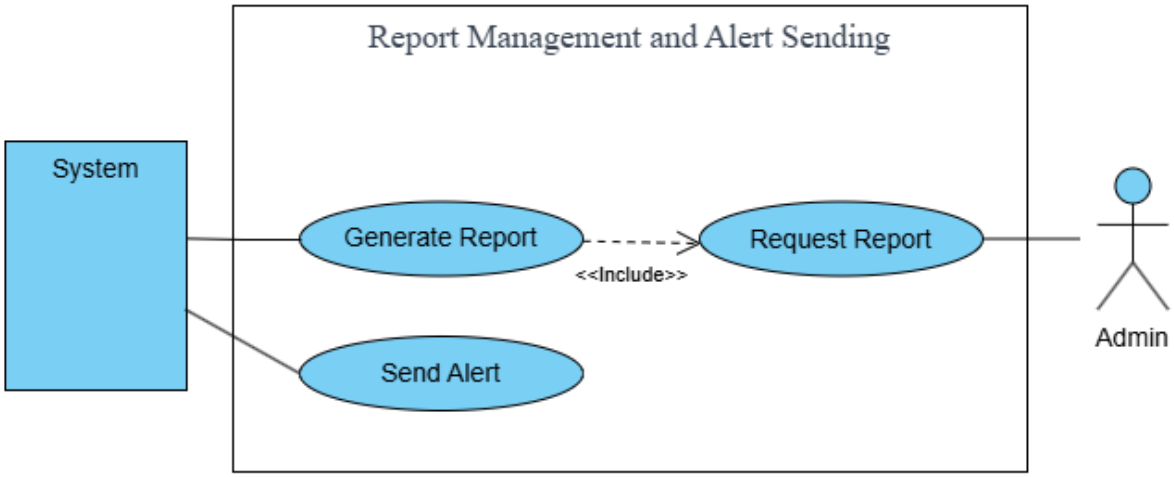

Figure 2.8: Use Case Diagram for Reporting and Alerts Sending

Table 2.8 provides a detailed description of the Generate Report use case, which is a core operation in the reporting module. This use case is important for providing insights into various aspects of the restaurant's operations, with a particular focus on identifying patterns and opportunities for reducing food wastage.

Table 2.8: Textual Description of Generate Report

Use Case Name	Generate Report
Actors	System, Admin
Description	This use case allows the system to automatically generate various types of reports for analysis.
Preconditions	1. The Admin is authenticated and logged into the system. 2. Data exists in the system for the selected report type and time period.
Basic Flow	1. Admin navigates to the Reporting section. 2. Admin selects the type of report. 3. Admin specifies the period for the report. 4. Admin selects any additional filters or parameters. 5. The system retrieves and processes the relevant data for the admin to analyze. 6. Admin has the option to export the report.

Use Case Name	Generate Report
Alternative Flows	5a. If no data exists for the selected parameters: 1. The system displays a message indicating that no data is available. 2. Admin can modify the parameters and try again. 6a. If exporting: 1. Admin clicks export (CSV format). 2. The system generates the export file. 3. The system provides the file for download.
Postconditions	1. The requested report is generated and displayed to the Admin. 2. If exported, a file containing the report data is provided.

Table 2.9 provides a detailed description of the Sending and Triggering Alerts use case, which automates notifications to manage inventory effectively.

Table 2.9: Textual Description of Sending and Triggering Alerts

Use Case Name	Sending and Triggering Alerts
Actors	System (Automated Process)
Description	This use case allows the system to automatically scan inventory every 24 hours and send notifications for low stock or near-expiring/expired items to Admin.
Preconditions	1. The system is operational and connected to the inventory database. 2. Inventory data is up-to-date with stock levels and expiration dates.
Basic Flow	1. The system initiates a daily scan of the inventory at a scheduled time. 2. The system checks for low stock items (below a predefined threshold). 3. The system checks for near-expiring or expired items (within 24 hours or past due). 4. For low stock items, the system sends a static notification to the Admin with no interaction options. 5. For near-expiring or expired items, the system sends a notification with two buttons: 'Move to Trash' and 'Extend Expiration (24h)'. 6. The Admin can mark any notification as read or delete it from the notifications list.
Alternative Flows	5a. If the Admin selects 'Move to Trash': 1. The system moves the item to the wastage log. 2. The notification is marked as resolved.

Use Case Name	Sending and Triggering Alerts
	5b. If the Admin selects 'Extend Expiration (24h)': 1. The system updates the expiration date by 24 hours. 2. The notification is marked as resolved. 6a. If no action is taken: 1. The notification remains in the list until marked as read or deleted.
Postconditions	1. Notifications are sent and displayed to the Admin based on inventory status. 2. Low stock items are flagged with static notifications. 3. Near-expiring or expired items are flagged with interactive notifications, with actions (trash or extend) completed if selected. 4. Notifications can be marked as read or deleted by the Admin.

2.4 Sequence Diagrams

This section presents sequence diagrams that depict the dynamic interactions and message flows between components of the Restaurant Management Software, illustrating key processes such as inventory management, order processing, and reporting. These diagrams, derived from the use case designs in this chapter, provide a clear visualization of how actors (e.g., admins, kitchen staff) and system modules collaborate to achieve functional requirements, serving as a foundation for implementation and testing in subsequent phases.

Figure 2.9 shows the sequence of interactions during the update of an inventory item. The diagram illustrates the flow from the user's action through the frontend components, to the backend API, and finally to the database. It includes the validation process, the update operation, batch management for items with expiration dates, and the generation of alerts if applicable.

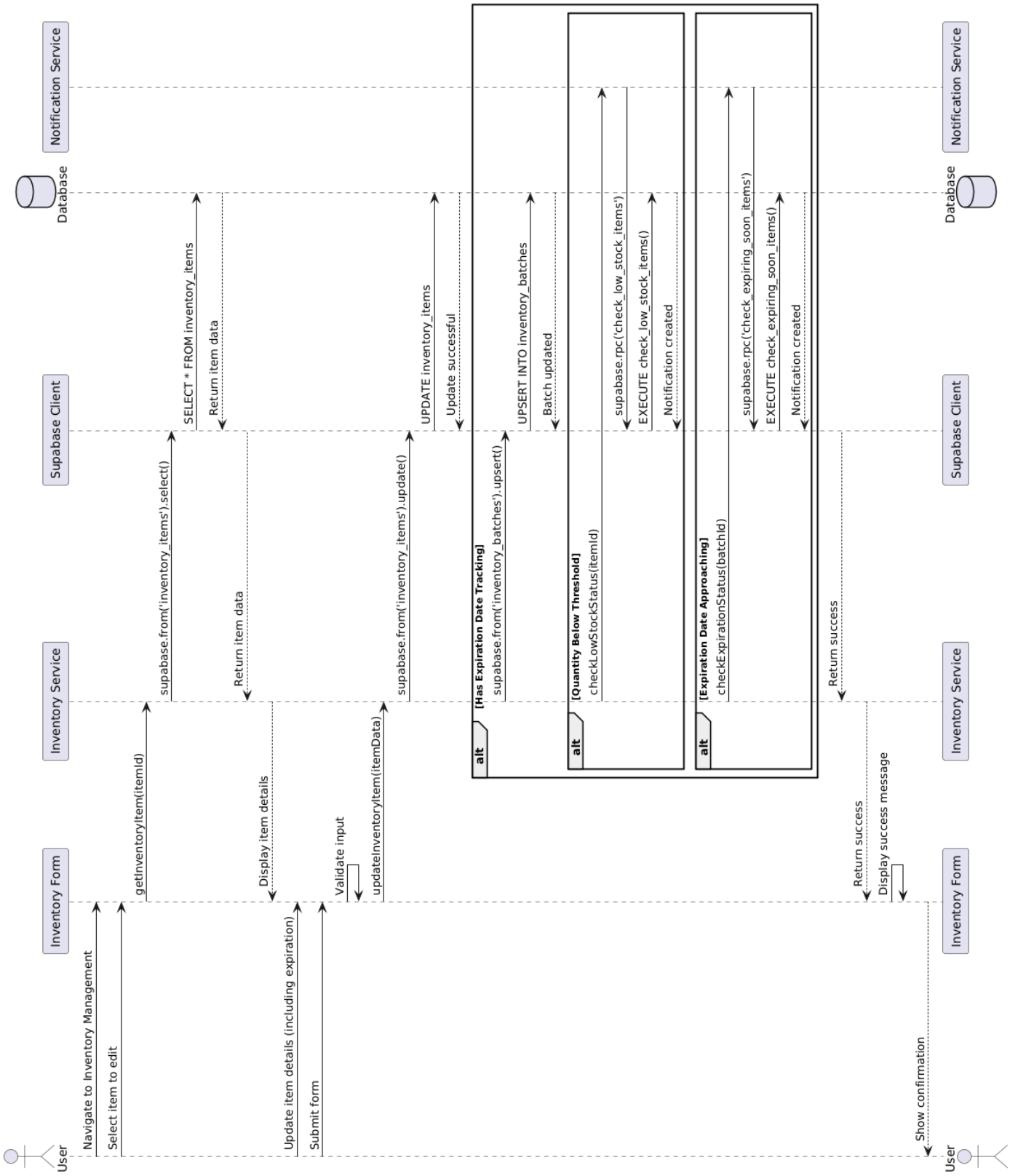

Figure 2.9: Sequence Diagram: Update Inventory Item

Figure 2.10 shows the sequence of interactions during the creation of an order. The diagram illustrates the flow from the user's action through the frontend components, to the backend API, and finally to the database. It includes the verification of inventory availability, the creation of the order record, the deduction of inventory based on the

recipes associated with the ordered menu items, the assignment of a customer via phone number lookup, and the application of any applicable loyalty discount.

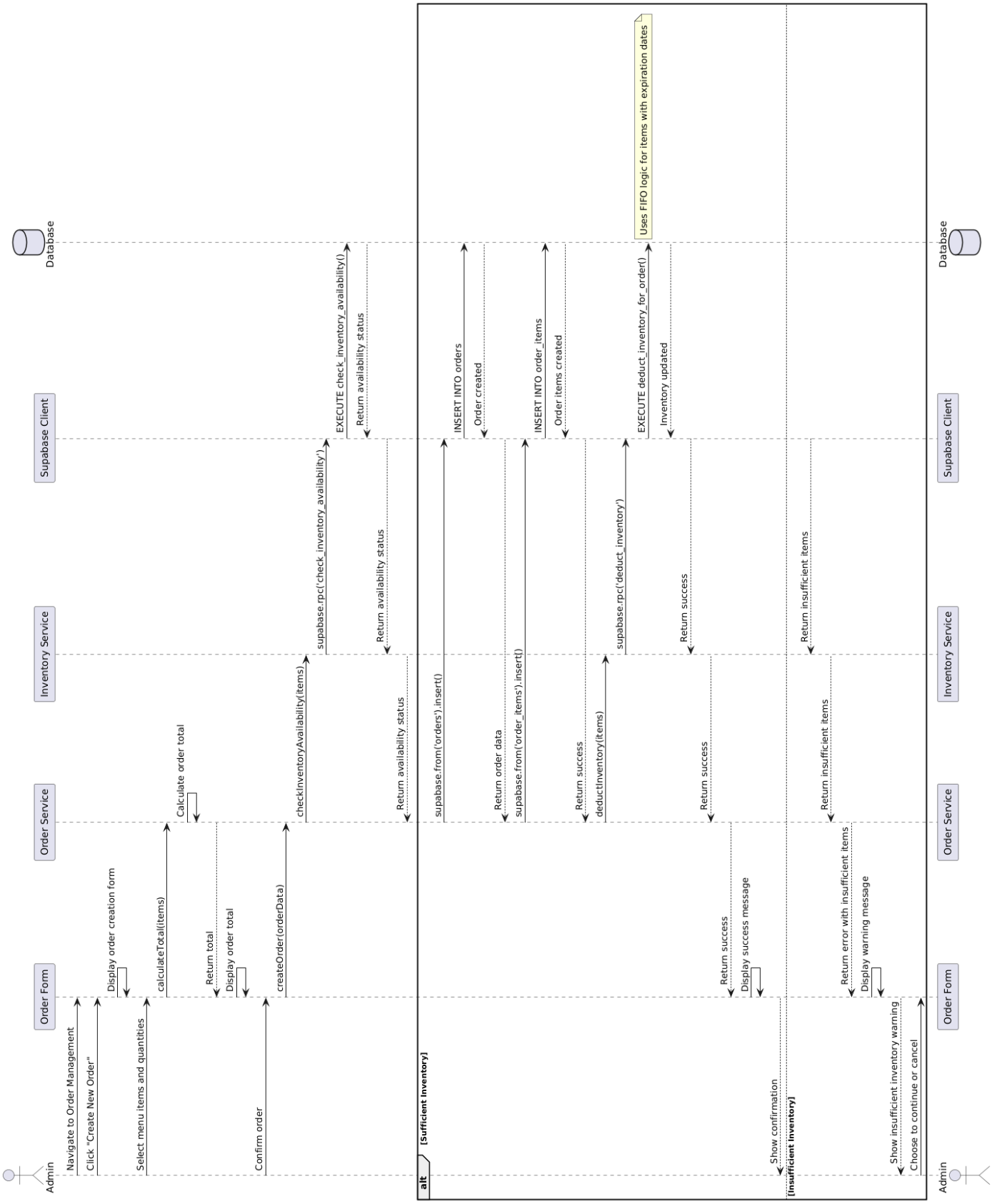

Figure 2.10: Sequence Diagram: Create Order

Figure 2.11 shows the sequence of interactions during the generation of a report. The diagram illustrates the flow from the user's request through the frontend components, to the backend API, and finally to the database for data retrieval. It includes the processing of the data, the generation of visualizations, the management of customer records, and the adjustment of loyalty program rules.

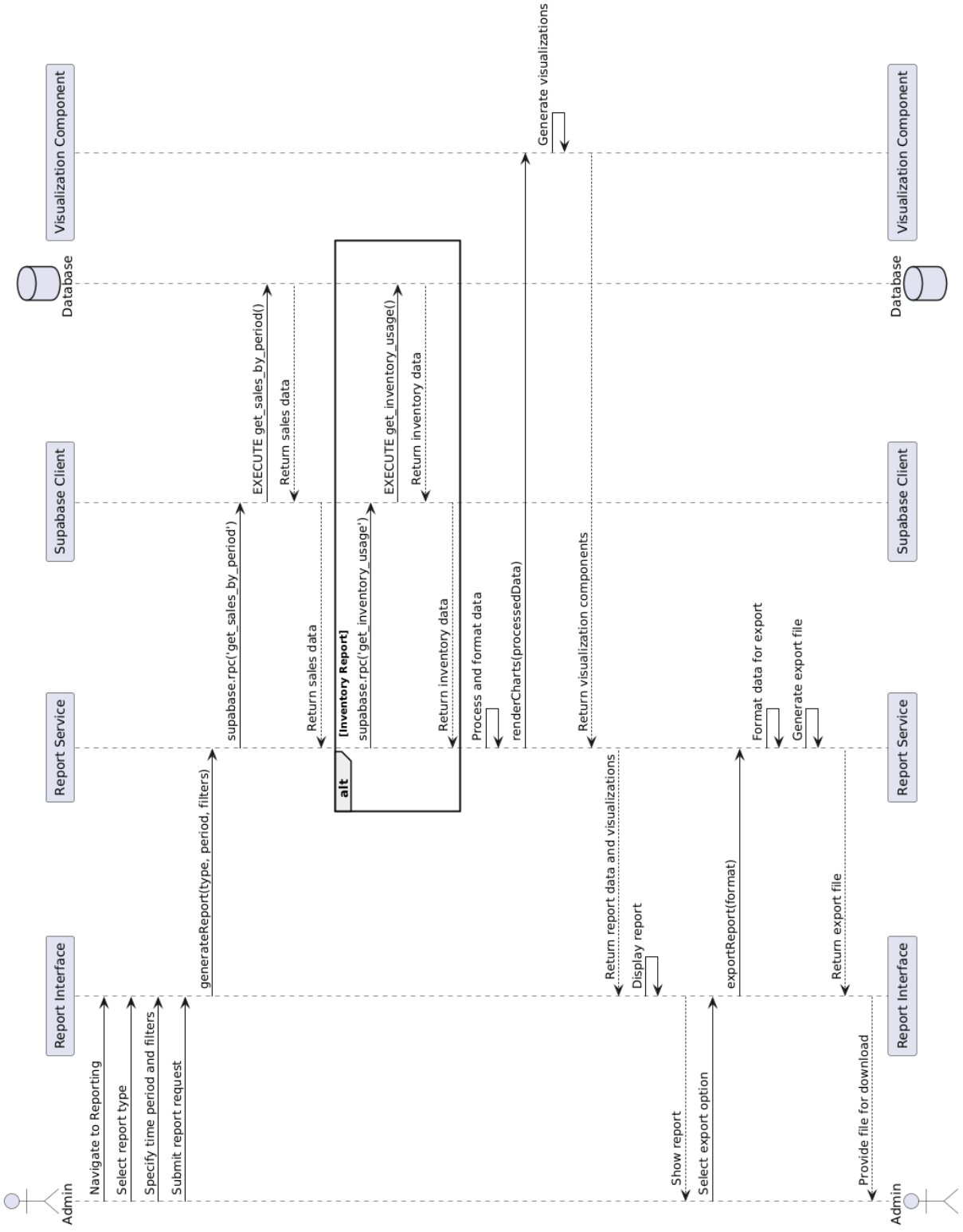

Figure 2.11: Sequence Diagram: Generate Report

2.5 Conclusion

The analysis phase established a solid foundation for the implementation phase by clearly defining the functionality of each module and the interactions between them. This thorough analysis was essential for ensuring that the system would effectively address the food wastage problem identified in Chapter 1.

The next chapter will detail the implementation of these features, including the technologies used, the development process, and the testing and validation procedures.

Chapter 3

Realization and Implementation Phase

3.1 Introduction

This chapter explains the realization and implementation phase of the Restaurant Management Software, covering all features that were analyzed in Chapter 2. During this phase, we turned the ideas and plans from earlier chapters into a working system that makes restaurant management easier, improves workflow, and increases the efficiency for owners. The software helps with tasks like tracking inventory, creating and managing orders, and providing insights for better decision-making, while also focusing on reducing food wastage, which is a major challenge identified in Chapter 1.

This chapter highlights the tools and technologies that we use, the user-friendly interfaces we designed, and the evaluation of the application to assess its effectiveness.

3.2 Technical Requirements and Constraints

This section presents the technological framework and limitations guiding the development of the Restaurant Management Software.

The technical requirements and constraints define the system's development and operational environment, ensuring compatibility with restaurant needs:

3.2.1 Hardware Environment

For our hardware environment, we utilized a laptop with the following specifications:

- **Model**: LENOVO IDEAPAD GAMING 3
- **Operating System**: Windows 10 Pro 64-bit
- **Processor**: AMD Ryzen 5-4600H, 3.30 GHz
- **Graphics**: Nvidia GeForce GTX 1650TI, 4 GB
- **Memory**: 16 GB
- **Disk Capacity**: 512 GB SSD

3.2.2 Software Environment:

This subsection describes the software solutions used in our implementation, including their responsibilities and explanations for use.

Initially, we conducted extensive research on existing backend software. We compared the characteristics of Supabase with PocketBase and Express.js + PostgreSQL. PocketBase, while lightweight and open-source, lacked the real-time capabilities and scalability needed for dynamic restaurant data, requiring additional configuration for authentication and APIs. Express.js + PostgreSQL offered flexibility but demanded significant manual setup for real-time features, authentication, and API endpoints, increasing development time and complexity. Supabase stood out for its Firebase-like simplicity, built-in real-time database, automatic API generation, and robust authentication, significantly accelerating development while meeting the project's need for a scalable, efficient backend. Its open-source nature further ensured cost-effectiveness and community support. Therefore, we selected to work with Supabase, a Backend-as-a-Service platform, for database, authentication, and API services (SupBas25). Figure 3.1 presents the logo of Supabase.

Figure 3.1: Supabase Logo

PostgreSQL, provided by Supabase, shall serve as the relational database (Con21). PostgreSQL ensures data integrity and supports complex queries, chosen for its reliability and performance in handling restaurant data. Figure 3.2 presents the logo of PostgreSQL.

Figure 3.2: PostgreSQL Logo

After that, we made a deep investigation of existing frontend programming software, such as React.js, Angular.js, Vue.js, and Nuxt.js, to deliver a responsive and dynamic interface. This review helps choose the frontend software that is relevant to the implementation of our project using Supabase. Existing frontend software (React.js, Angular.js, and Vue.js) presents a lightweight nature and flexibility in building component-based interfaces. In contrast, Nuxt.js addresses specific project needs. More specifically, Nuxt.js offers several advantages (Nuxt25), including a built-in server-side rendering (SSR) for improved SEO and faster initial page loads, automatic code splitting for optimized performance, and simplified routing with a file-based structure that reduces manual configuration. Additionally, Nuxt.js provides pre-configured features like middleware for authentication and a robust module ecosystem, which streamlined integration with Supabase and

enhanced scalability for our restaurant management application. These benefits aligned better with the project's goals of performance optimization, user experience, and rapid development for small to medium-sized businesses. Therefore, we chose to work with Nuxt.js. Figure 3.3 presents the logo of Nuxt.js.

Figure 3.3: Nuxt.js Logo

To achieve efficient project tracking, we use Cursor as the primary code editor (Cursor25) and GitHub for version control (Git25). Cursor is an AI-powered editor that enhances productivity with code suggestions and debugging support, chosen for its efficiency in web development tasks. Figure 3.4 presents the Cursor logo. GitHub provides robust version control and collaboration features, selected for its seamless integration with team workflows and project tracking. Figure 3.5 presents the logo of GitHub.

Figure 3.4: Cursor Logo

Figure 3.5: GitHub Logo

Playwright Test, provided by Microsoft, shall be used for end-to-end testing (PWright25). Playwright supports cross-browser testing and automation, selected for its reliability in ensuring the application's functionality across different environments. Figure 3.6 presents the logo of Playwright.

Figure 3.6: Playwright Logo

Docker shall be used for containerization to ensure consistent development and testing environments (Docker25). Docker allows packaging the application with its dependencies

into containers, chosen for its ability to replicate production-like environments locally, simplify dependency management, and facilitate seamless deployment workflows. Figure 3.7 presents the logo of Docker.

Figure 3.7: Docker Logo

3.3 Implementation

This section outlines the development of the Restaurant Management Software's core modules—Inventory Management, Recipe Management, Menu Management, Order Management, Kitchen Interface, and Reporting—transforming the designs from Chapter 2 into a functional system to address the operational challenges identified in Chapter 1.

3.3.1 Inventory Management

The Inventory Management module was built to provide tools for tracking and managing food stock, with a focus on reducing food waste, a key goal from Chapter 1. This section details the evolution of the module, from its initial beta version to the final implementation, highlighting development stages, refactoring efforts, functional enhancements, and user interface improvements.

Initial Beta Version Development

The initial beta version of the Inventory Management module was developed to establish core functionality with a minimal feature set. The user interface (UI) was rudimentary, featuring basic tables to list inventory items with fields for name, quantity, and cost, but lacking advanced features like predefined quantity units, expiration date tracking, or batch logic. Filtering and sorting options were absent, and the design was not user-friendly, with a plain layout, limited color coding, and no interactive elements like alerts for low stock. Figure 3.8 illustrates the beta UI, showing a simple table listing items without actionable insights or visual cues for usability.

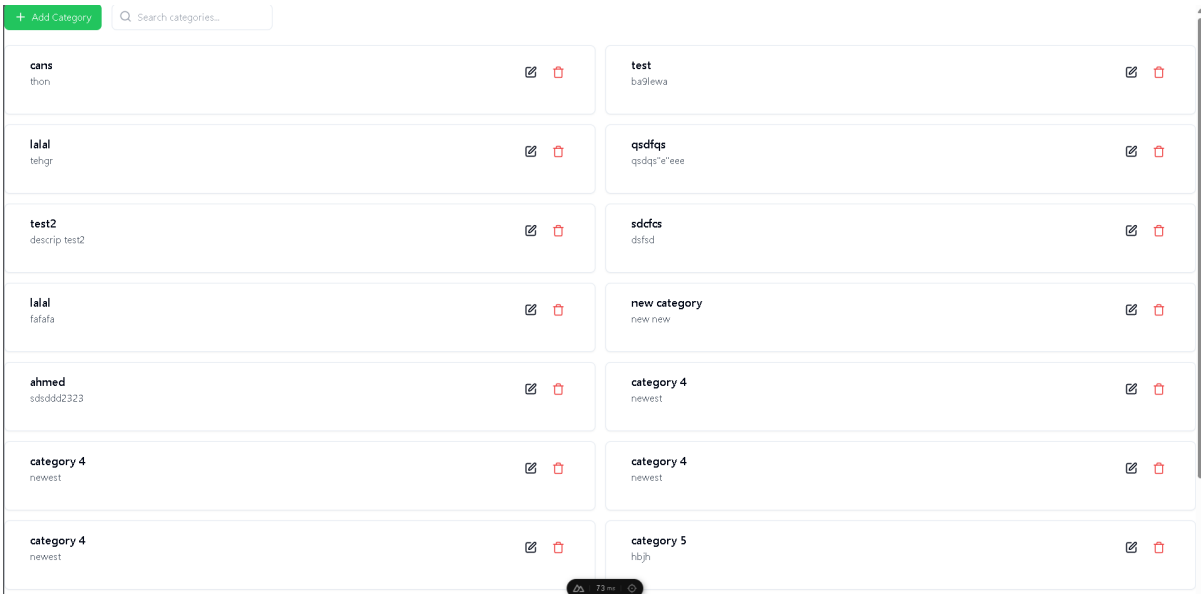

Figure 3.8: Initial Beta Inventory Interface

The beta version also lacked data validation, allowing inconsistent entries (e.g., negative quantities), and had no mechanisms for tracking wastage or managing expiration dates, limiting its effectiveness in addressing food waste.

Major Refactoring of Inventory Pages

Following feedback on the beta version, a major refactoring of the Inventory Management pages was undertaken to improve the project's structure and code organization. Initially, the codebase was disorganized, with tightly coupled logic, duplicated code across pages, and inconsistent naming conventions, making maintenance and scalability challenging. The refactoring process focused on restructuring the project into a modular architecture, organizing the inventory pages into well-defined directories (e.g., components, pages, and utilities) to enhance clarity and maintainability. We introduced custom and reusable components, such as `Buttons` `Inputs` and many others, which reduced redundancy and ensured consistency across the module. Additionally, we implemented layouts for every type of page to standardize the user interface structure. In addition, this structural overhaul transformed the previously messy codebase into a clean, organized, and scalable foundation, facilitating the addition of new features.

Functional Enhancements

Significant functional enhancements were introduced post-refactoring to improve the module's effectiveness. Predefined quantity units were added, allowing users to select units like "kg" or "L" with dynamic steps (e.g., 0.1 for kg), ensuring accurate stock tracking. Expiration date management was implemented, introducing batch logic to track inventory batches with expiration dates, supported by the 'inventory-batches' table. This enabled features like First In, First Out (FIFO) inventory deduction via the 'deduct-inventory' function and expiration alerts using the 'notifications' table, directly addressing the food waste challenge. Additionally, wastage tracking was integrated, automatically recording expired items in the 'wastage' table, enhancing data-driven decision-making for restaurant managers.

User Interface Overhaul

The user interface was overhauled to address the beta version's shortcomings, resulting in a more intuitive and efficient design. The revamped Inventory Management screens now allow admins to view, add, update, and remove items, categories, and batches, as shown in Figure 3.9. Compared to the beta version, the main screen provides a clear overview of stock levels with color coding to highlight low stock or near-expiring items, and includes filtering and sorting options for quick access—features absent in the beta UI.

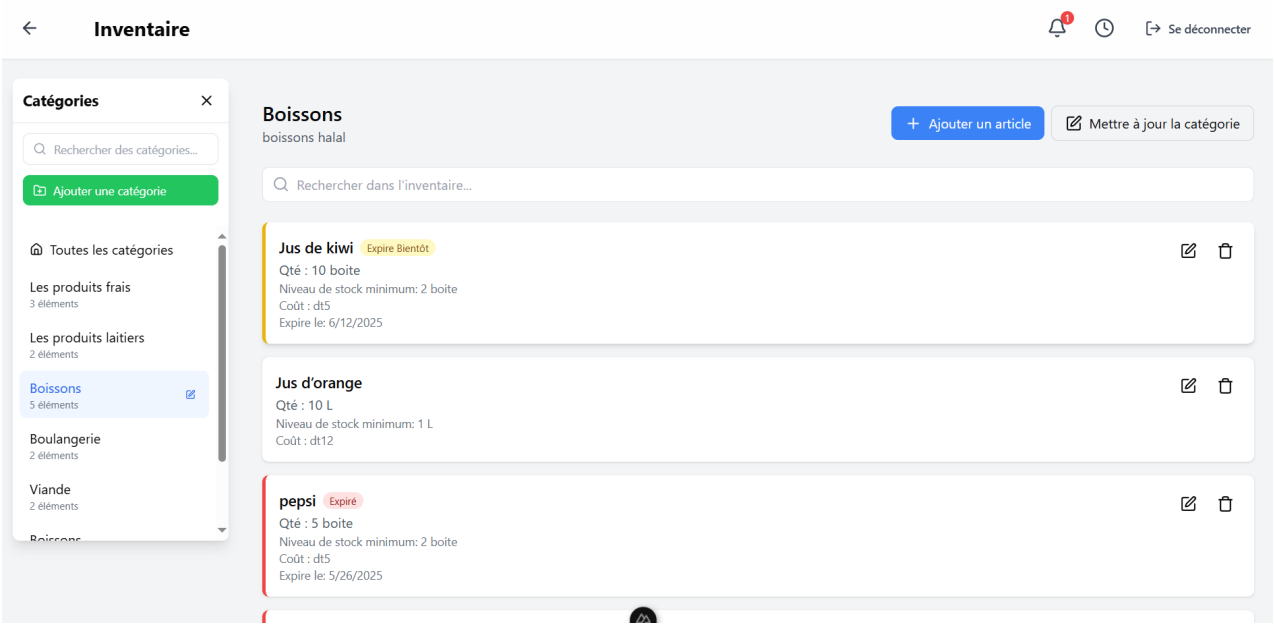

Figure 3.9: Inventory Management Interface

Figure 3.10 presents the screen for editing an inventory item, enabling updates to name, category, unit, amount, and cost, with input validation and error messages—improvements over the beta's lack of validation.

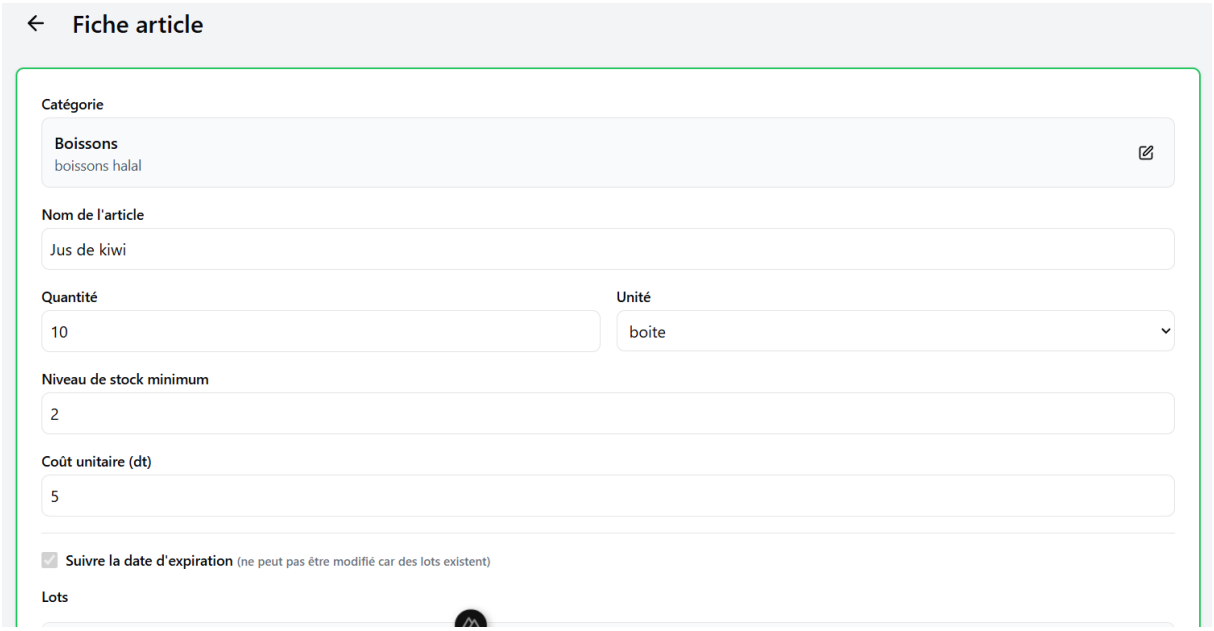

Figure 3.10: Inventory Item Edit Form

Expiration date management was integrated into the UI, with a dedicated screen (Figure 3.11) highlighting soon-to-expire items and providing tools for batch management and date extensions, a significant upgrade from the beta version's absence of such features. Key enhancements include batch tracking via the 'inventory-batches' table, expiration alerts using the 'notifications' table, FIFO deduction with 'deduct-inventory', date extensions via 'extend-batch-expiration', and automatic wastage recording in the 'wastage' table.

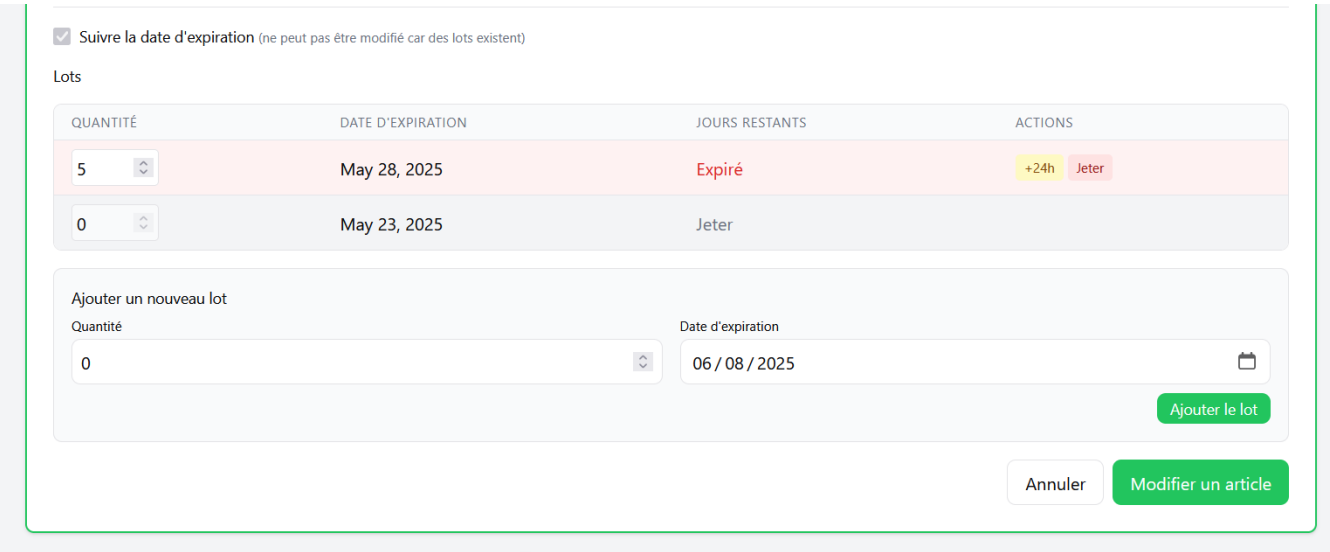

Figure 3.11: Expiration Date Management Interface

3.3.2 Recipe Management

The Recipe Management module enables users to create and manage recipes with specific ingredients and amounts, linking inventory to menu items for accurate tracking.

Recipe Data Model

The recipe data includes recipes, ingredients, amounts, and preparation steps, following Chapter 2's schema:

- **Recipes**: Details like name, description, and preparation steps in the 'recipes' table.
- **Recipe Ingredients**: Linking recipes to inventory items with amounts and units via 'recipe-items'.
- **Recipe Categories**: Grouping recipes for easier management in a separate table.
- **Preparation Steps**: Storing steps as text in the 'recipes' table.

Recipe Management Interface

The Recipe Management screens allow users to view, add, update, and remove recipes, categories, and ingredients. As shown in Figure 3.12, the main screen lists all recipes with filtering options, displaying name, description, and amounts.

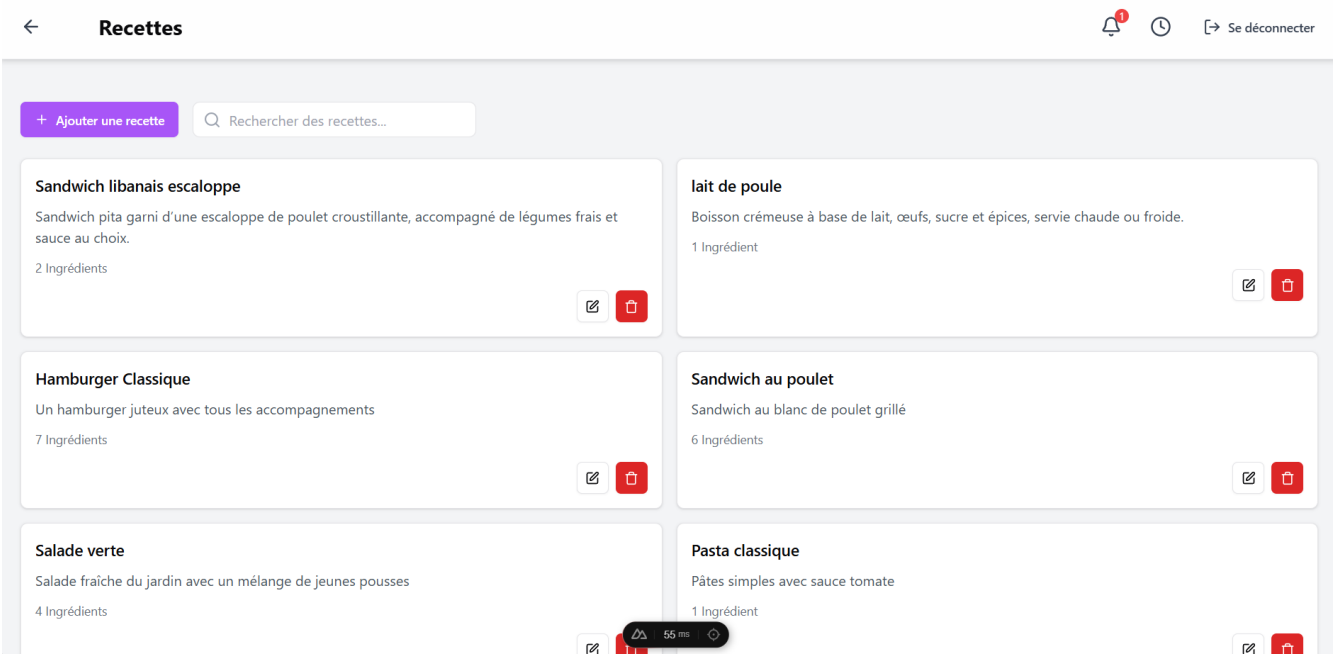

Figure 3.12: Recipe Management Interface

Figure 3.13 displays the screen for adding a recipe, where users input name, category, description, preparation time, and ingredients, with validation and error messages.

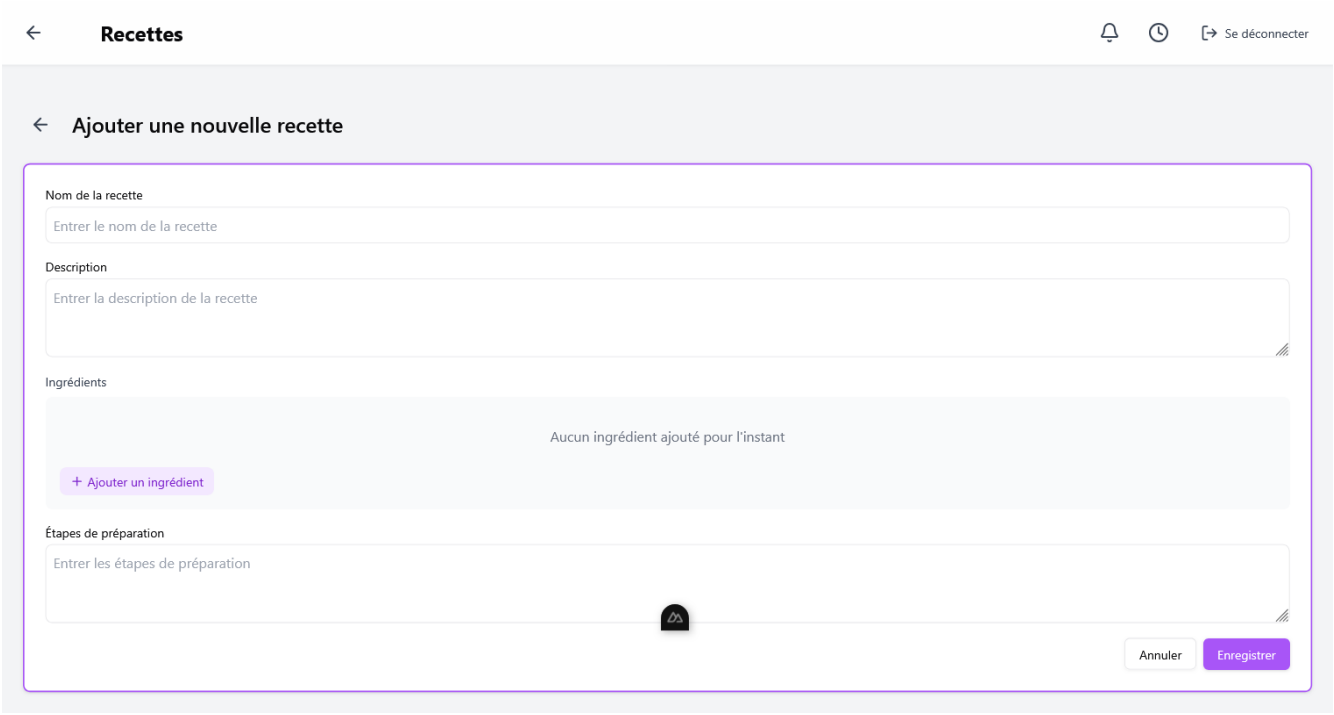

Figure 3.13: Recipe Creation Form

3.3.3 Menu Management

The Menu Management module allows users to create and manage menu items linked to recipes or inventory items, facilitating the connection between recipes and orders as well as inventory items and orders.

Menu Data Model

The menu data includes menu items, recipes, prices, and categories, as per Chapter 2's schema:

- **Menu Items**: Details like name, category, description, price, and linked recipe or inventory item in the 'menu-items' table.
- **Menu Categories**: Grouping items for easier management via 'menu-categories'.

Menu Management Interface

The Menu Management screens enable users to view, add, update, and remove menu items, categories, and prices. As illustrated in Figure 3.14, the main screen lists all menu items with filtering options, showing recipes, preparation time, notes, sources and prices.

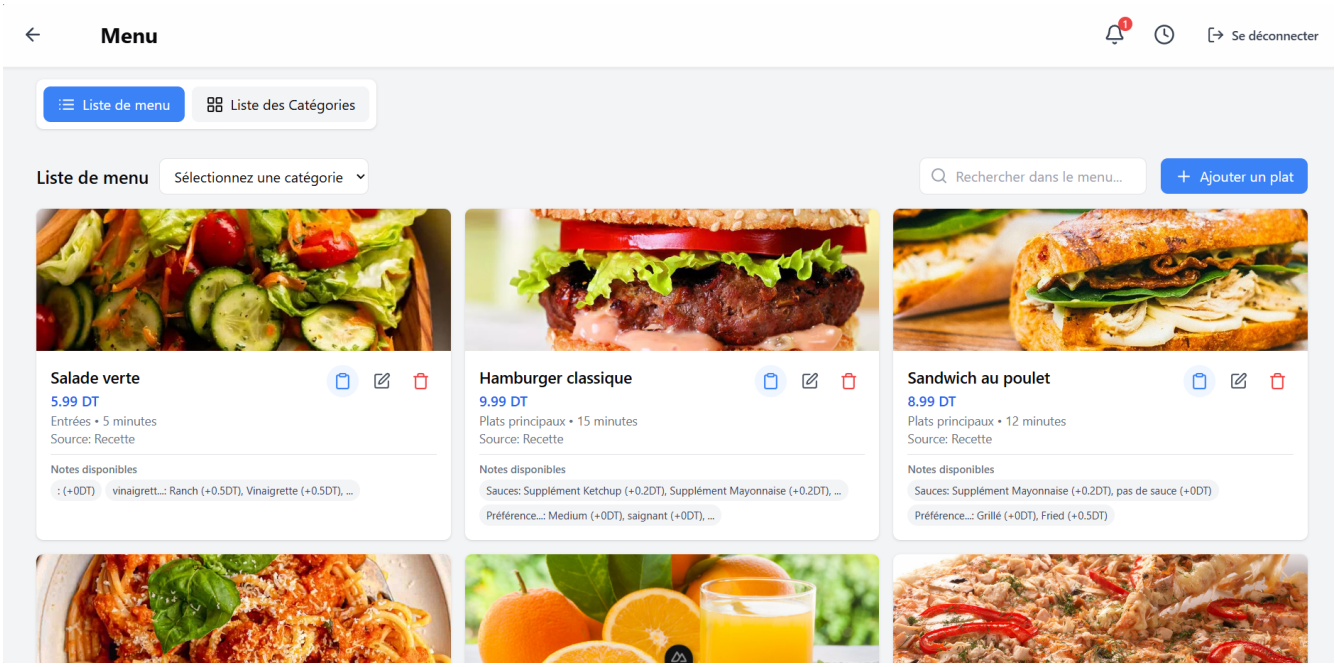

Figure 3.14: Menu Management Interface

Figure 3.15 presents the screen for adding a menu item, allowing input of name, category, description, price, source, and source item with validation and error messages.

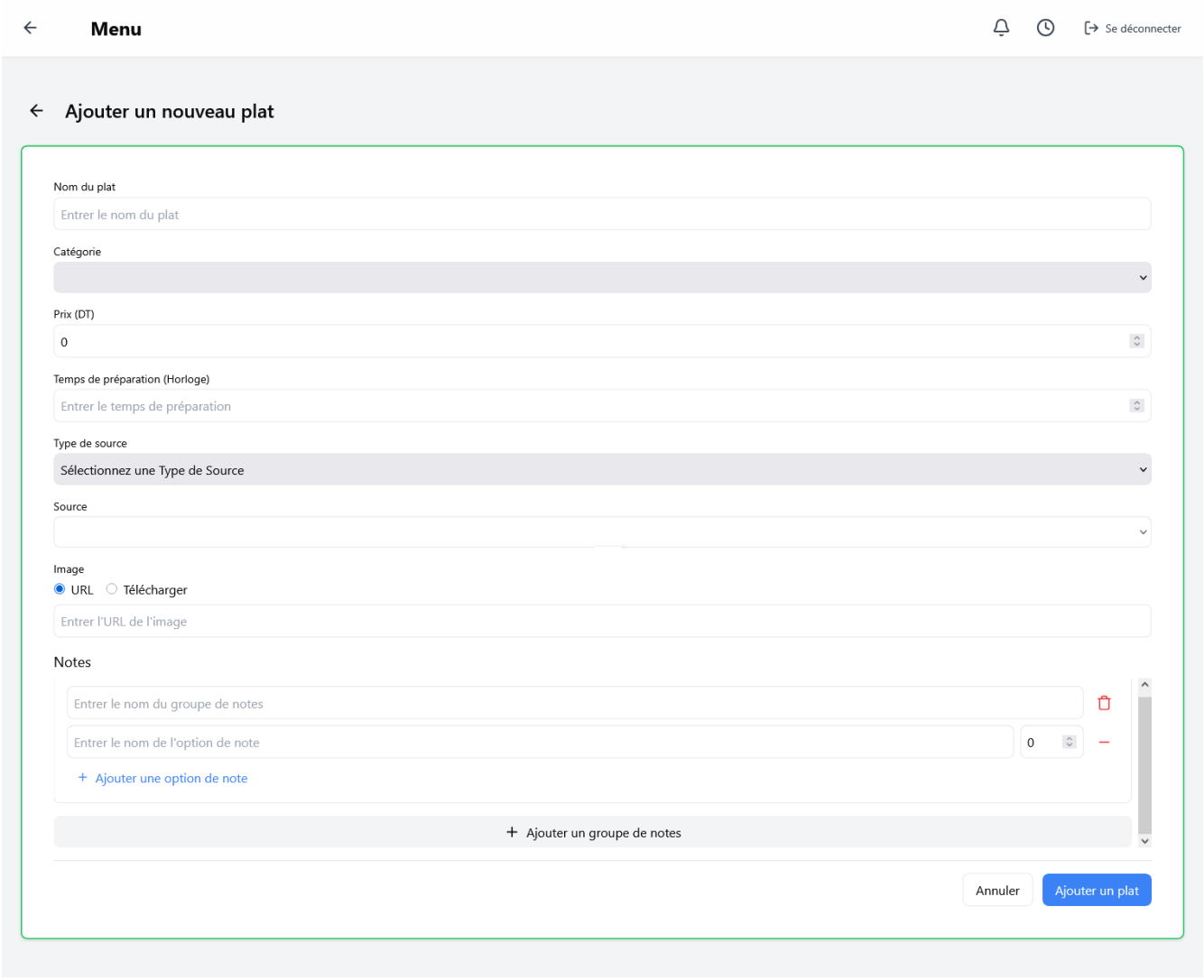

Figure 3.15: Menu Item Creation Form

3.3.4 Order Management

The Order Management module streamlines order creation and tracking, automatically updating inventory based on recipes, and includes customer and loyalty features.

Order Data Model

The order data includes orders, items, amounts, and statuses, following Chapter 2's design:

- **Orders**: Details like date, status, total cost, notes, customer info, and discounts in the 'orders' table.
- **Order Items**: Linking orders to menu items with amounts and costs via 'order-items'.
- **Order Status History**: Tracking status changes with dates and users in a separate table.

Order Management Interface

The Order Management screens allow users to create, view, and manage orders. As shown in Figure 3.16, the main screen lists orders, displaying items, amounts, costs, customer info, and status history.

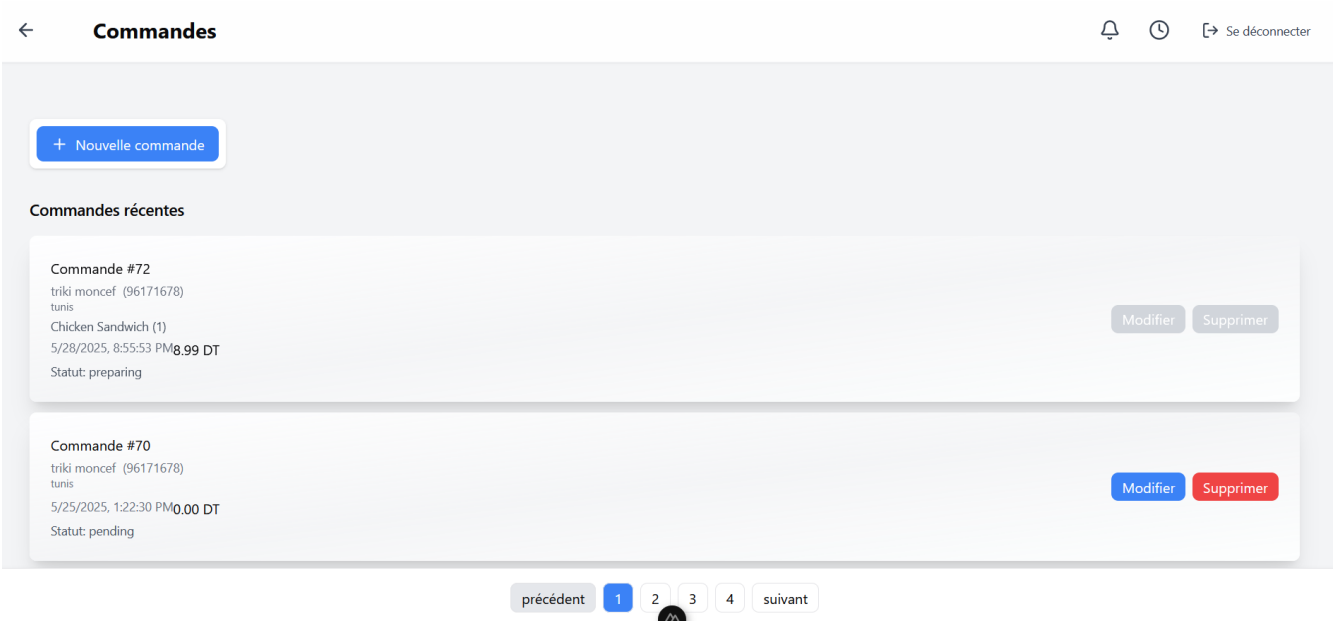

Figure 3.16: Order Management Interface

Figure 3.17 displays the order creation screen, where users select menu items, set amounts, custom notes (eg.,Fried) assign customers by phone number, and apply loyalty discounts, with real-time cost calculation.

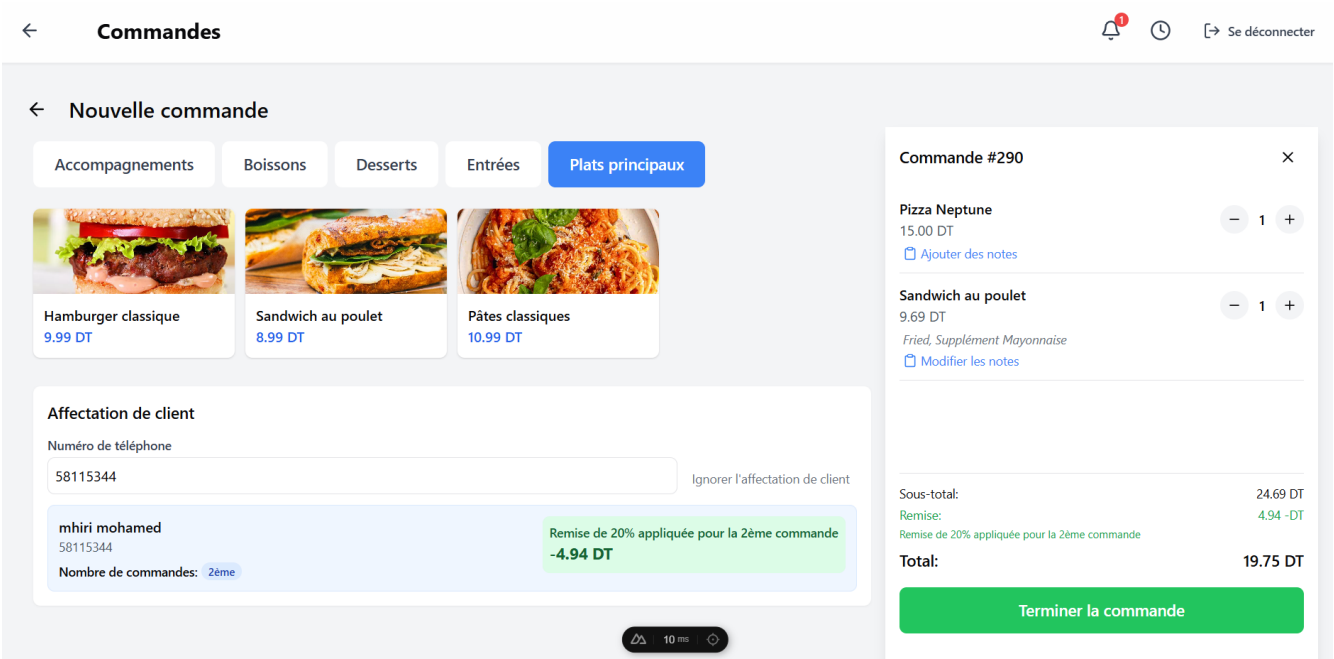

Figure 3.17: Order Creation Interface

Inventory Deduction

We implemented automatic inventory updates for orders:

- **Getting Recipes**: Retrieving recipes for order menu items.
- **Calculating Ingredients**: Determining required ingredient quantities.
- **Checking Stock**: Ensuring sufficient stock availability.
- **Using Oldest Items First**: Following First In, First Out via ‘deduct-inventory‘.
- **Recording Changes**: Logging stock changes in ‘inventory-transactions‘.

Customer and Loyalty Integration

We added customer management and loyalty features:

- **Customer Assignment**: Linking customers to orders via phone number with ‘get-customer-by-phone‘.
- **Loyalty Discount Application**: Applying discounts (e.g., 10% off on 5th order) using ‘calculate-loyalty-discount‘.
- **Updating Customer Stats**: Updating order totals and spending in ‘customers‘ via ‘update-customer-order-stats‘.

3.3.5 Kitchen Interface

The Kitchen Interface module provides kitchen staff with a simple interface to view and manage orders for preparation, ensuring efficiency.

Kitchen Interface Design

The Kitchen Interface is designed for simplicity in a busy environment. As illustrated in Figure 3.18, it displays pending and in-progress orders, listing menu items, amounts, preparation steps, and wait times, with large buttons and a drag and drop feature for status updates to minimize errors, also the possibility to move order items to trash.

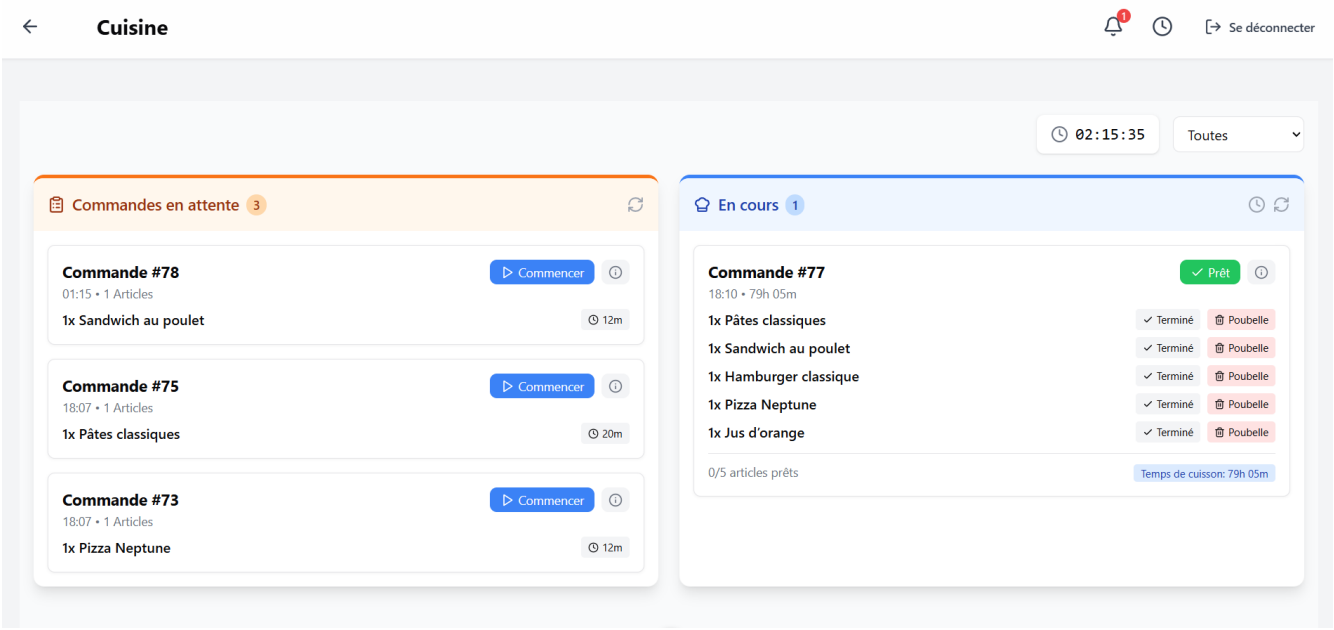

Figure 3.18: Kitchen Interface

Order Status Management

We implemented order status tracking for the kitchen:

- **Status Steps**: Tracking stages like "Pending," "In Progress," "Ready," and "Completed."
- **Status Updates**: Allowing staff to update order or item statuses.
- **Time Tracking**: Monitoring time spent in each stage to identify delays.
- **Throwing Away Items**: Recording discarded items in ‘wastage‘ via ‘mark-item-as-wastage‘.

3.3.6 Reporting

The Reporting module provides insights into restaurant performance, focusing on reducing food waste, and includes customer and loyalty management tools.

Reporting Data Model

The reporting data aggregates information for analysis:

- **Sales Data**: Total revenue, popular items, and trends from ‘orders‘.
- **Inventory Usage Data**: Usage, costs, and waste from ‘inventory-transactions‘ and ‘wastage‘.
- **Menu Performance Data**: Popularity and profitability from ‘menu-items‘ and ‘orders‘.
- **Wastage Data**: Waste details and costs in ‘wastage‘.
- **Customer Data**: Order history and spending in ‘customers‘.

Reporting Interface

The Reporting screens enable report creation and visualization. As shown in Figure 3.19, the dashboard provides an overview with charts and tables for sales, inventory, menu performance, wastage, and customer insights.

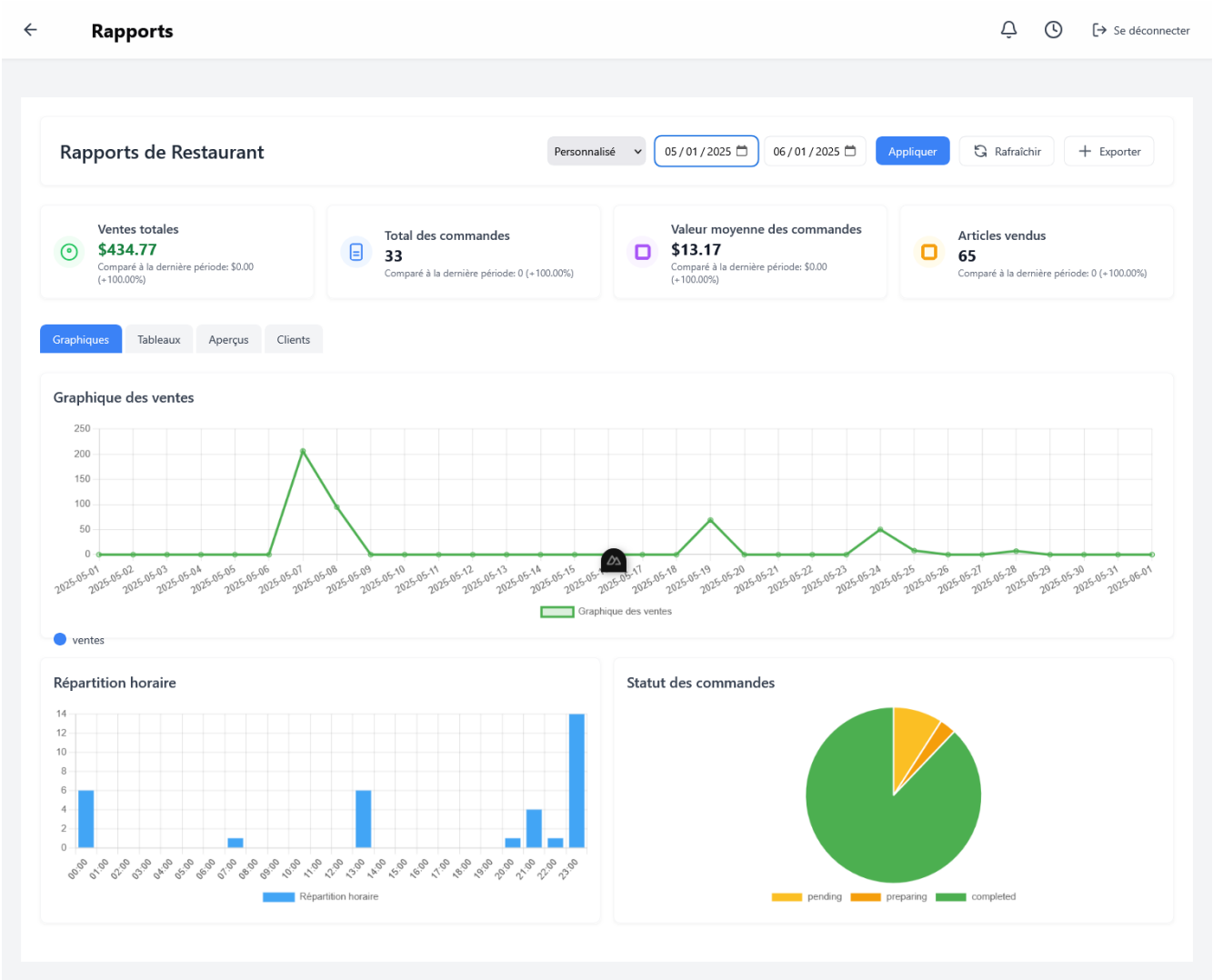

Figure 3.19: Reporting Dashboard

Wastage Analysis

We implemented a feature to analyze food waste:

- **Tracking Waste**: Recording waste from expiration or errors in ‘wastage‘.
- **Looking at Patterns**: Analyzing waste trends over time.

Figure 3.20 provides the Peak Hours, Inventory Alerts, Recommendations and Wastage report.

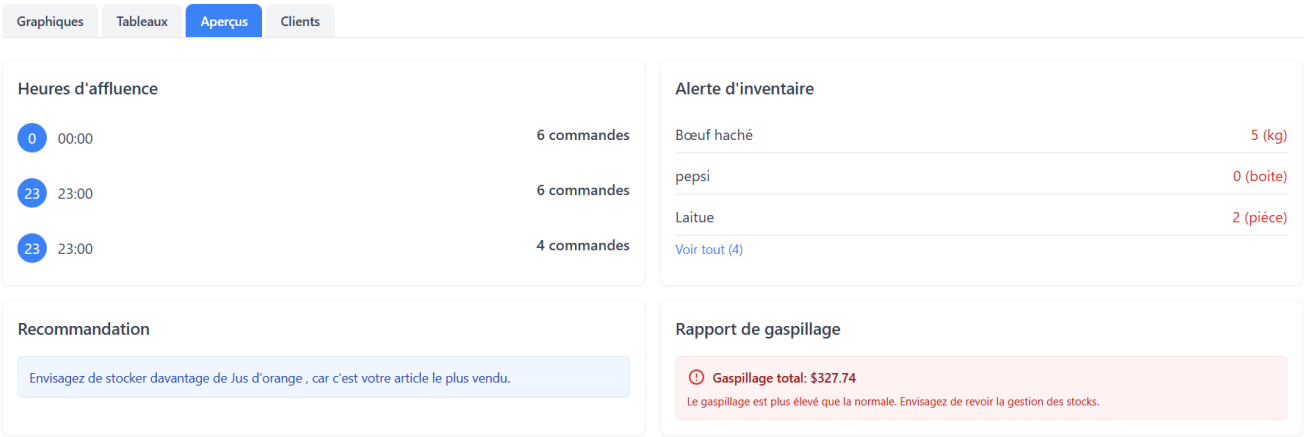

Figure 3.20: Insights Interface

Customer and Loyalty Management

We added customer and loyalty management features:

- **Viewing Customers**: Listing customers and top spenders via 'get-top-customers'. Figure 3.21 illustrates the interface for managing customers, displaying a table with customer details and top spenders.
- **Updating Customers**: Modifying customer details in 'customers'.
- **Managing Loyalty Programs**: Setting rules (e.g., 10% off on 5th order) in 'loyalty-settings'. Figure 3.22 shows the interface for managing loyalty settings, highlighting the configuration options for loyalty rules.

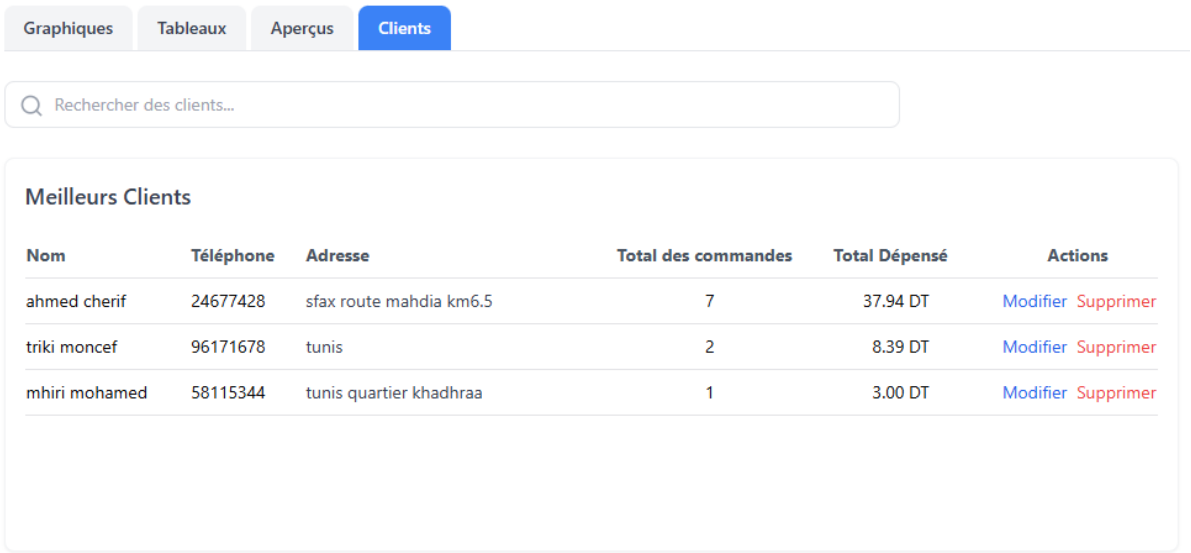

Figure 3.21: Interface for Managing Customers

Figure 3.22: Interface for Managing Loyalty Settings

Alerts Sending and Notification Tray

This enhancement extends the system's operational efficiency by keeping Admin informed about inventory status. Figure 3.23 illustrates the notification tray interface, showcasing the layout and interaction options for managing alerts.

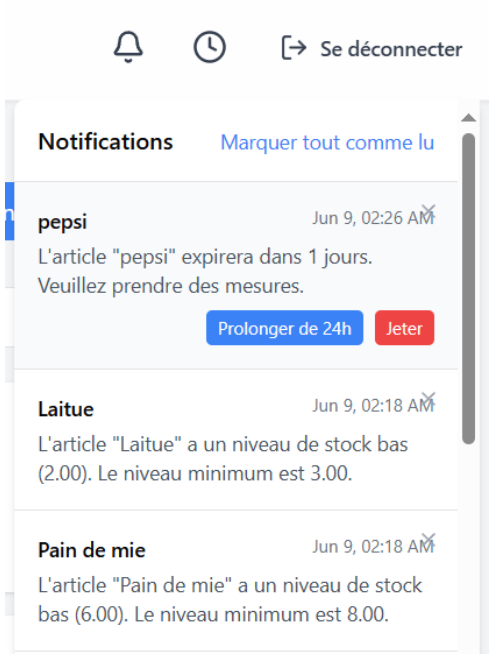

Figure 3.23: Notification Tray Interface

3.4 Testing and Validation

This section consolidates the testing and validation efforts for all modules—Inventory Management, Recipe Management, Menu Management, Order Management, Kitchen Interface, and Reporting—to ensure the system's accuracy, reliability, and usability while

avoiding redundancy in evaluation approaches.

3.4.1 Manual Verification Procedures

Manual verification procedures were critical in ensuring the system's stability and functionality before automated testing, addressing both version control challenges and initial feature validation.

Resolution of Version Control Conflicts

During development, we encountered significant version control conflicts due to the creation of multiple branches for various features, some built on the main branch and others on secondary branches, leading to complex merge issues. Figure 3.24 visualizes the initial source control status, illustrating overlapping branches and conflicts.

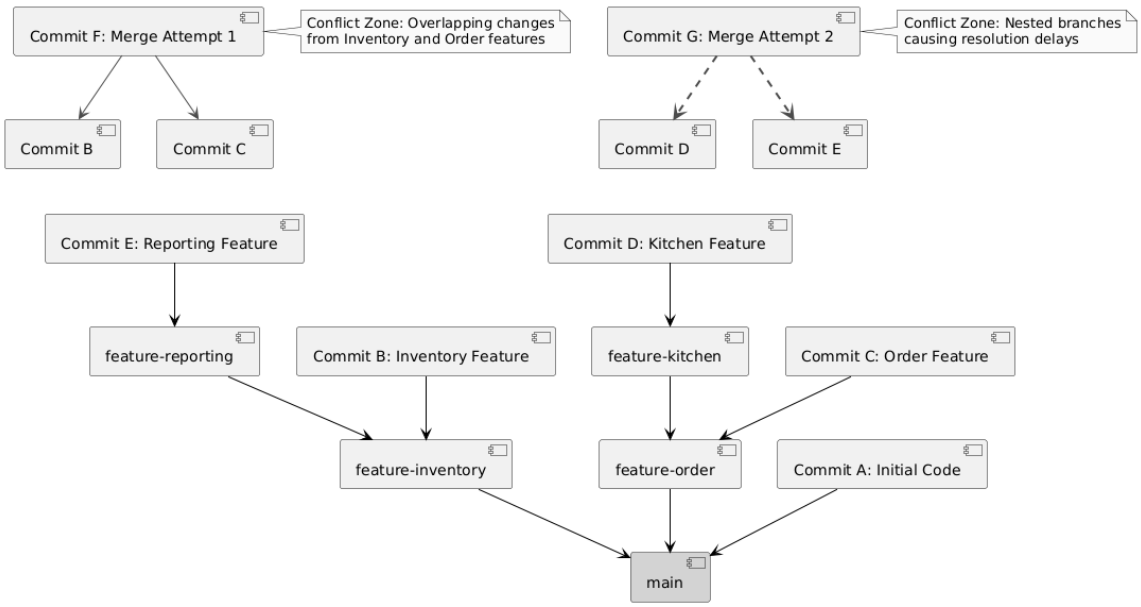

Figure 3.24: Initial Version Control Conflict Status

Initial Conflict Resolution: One-Day Effort

The initial approach to resolving these conflicts involved manually merging branches and resolving discrepancies line-by-line, a highly time-intensive process. This method required over a day of effort, involving detailed code reviews and frequent communication among team members, highlighting the inefficiency and potential for errors in such a labor-intensive resolution.

Optimized Conflict Resolution Using Git Cherry-Pick, Squash, and Rebase: Under One Hour

To streamline the process, we adopted an optimized solution using Git cherry-pick, squash, and rebase techniques. Cherry-pick allowed selective integration of specific commits from conflicting branches, squash combined multiple commits into a single, coherent change, and rebase reorganized the commit history for clarity. This approach reduced resolution time to under one hour, minimizing errors and maintaining a clean project history. Figure

3.25 illustrates the streamlined workflow, showcasing the power of these Git commands in resolving conflicts efficiently.

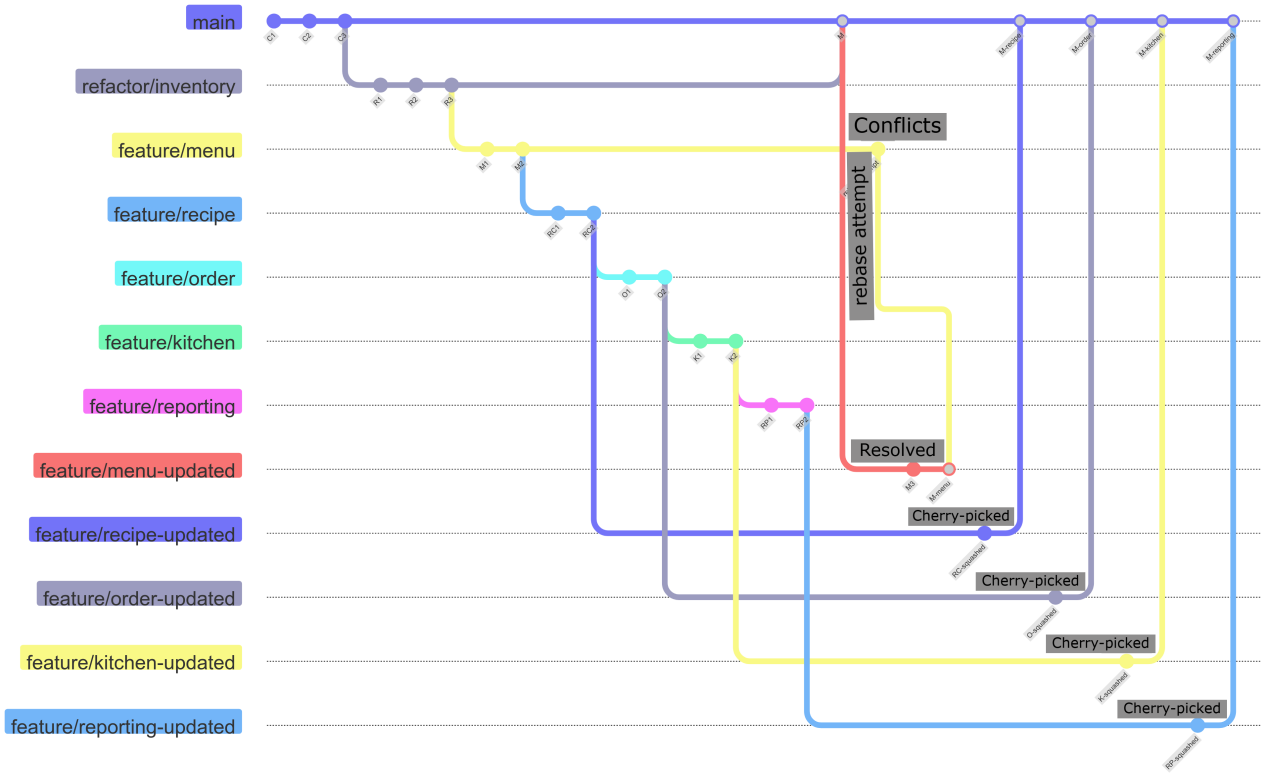

Figure 3.25: Optimized Version Control Workflow

Manual Testing and Documentation

Before initiating end-to-end testing with Playwright, manual testing was conducted to identify and fix bugs, ensuring core functionalities worked effectively. This process involved step-by-step validation of features across all modules, with the README.md file (e.g., inventory feature testing) serving as an example of detailed documentation. Key points included verifying inventory item creation, editing, and deletion, as well as expiration date alerts and order deductions, with results logged to track issues like missing toast messages or UI responsiveness, facilitating iterative improvements and ensuring a stable foundation for automated tests.

3.4.2 End-to-End System Validation

We conducted comprehensive testing across all modules using the following methods, enhanced by a dedicated testing environment:

To enhance validation, we created a dedicated copy of the Supabase backend for testing, configured with a separate database instance to simulate production conditions without affecting live data. This environment was set up using Docker containers, ensuring consistency across development, testing, and deployment phases, and allowing us to replicate real-world scenarios (e.g., high transaction volumes) for thorough validation. These tests confirmed that all modules meet the specifications from Chapter 2, delivering a

reliable system that enhances restaurant operations, reduces food waste, and supports customer engagement through loyalty programs.

3.5 Deployment and Maintenance

This section outlines the deployment strategy and ongoing maintenance plan for the Restaurant Management Software, leveraging modern containerization to ensure scalability and reliability.

The deployment process utilized Docker to containerize the application, packaging the Nuxt.js frontend and the Supabase backend, which includes an embedded PostgreSQL database, into isolated containers (Docker25). This approach ensured consistent environments across development, testing, and production, mitigating issues like dependency conflicts or configuration mismatches. Docker Compose was employed to orchestrate multiple containers, defining services, networks, and volumes in a single ‘docker-compose.yml‘ file, including a separate PostgreSQL backup volume for data persistence, enabling seamless scaling and easy updates. Maintenance involves regular container updates to address security patches, monitoring resource usage with Docker’s built-in tools, and backing up the PostgreSQL database to the backup volume to prevent data loss, ensuring long-term sustainability for restaurant users. Figure 3.26 illustrates the containerized deployment architecture, showing the interaction between the frontend, the Supabase backend with its embedded PostgreSQL database, and the backup volume.

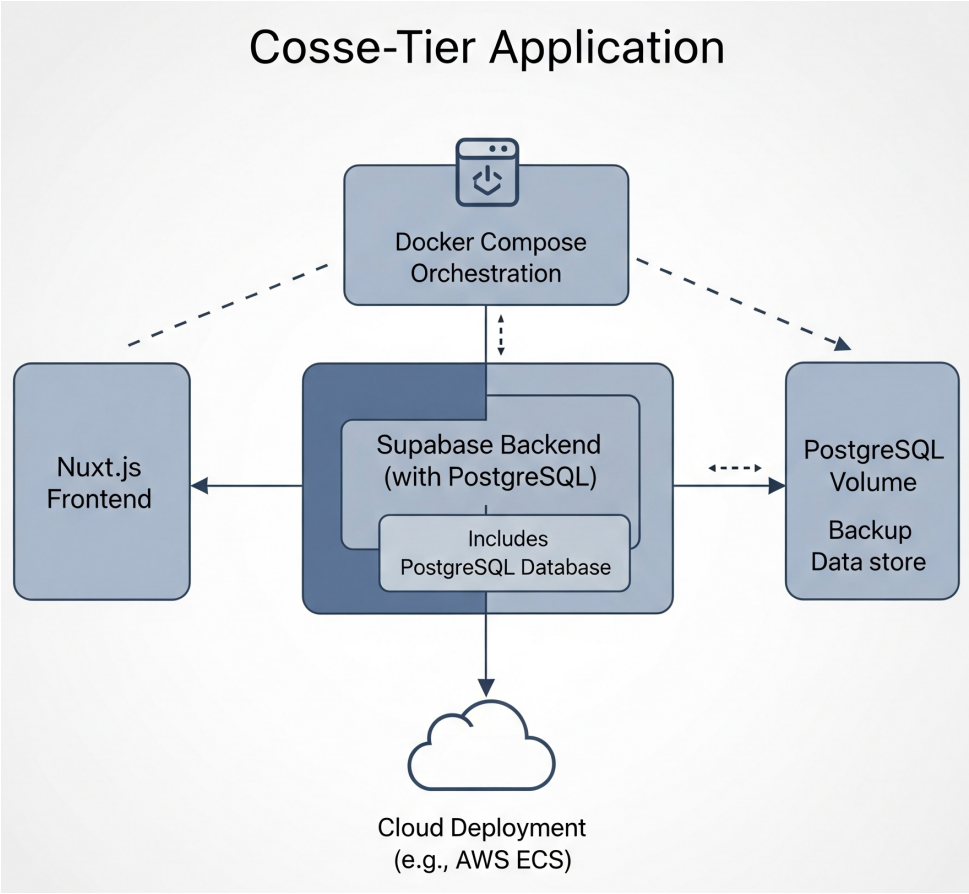

Figure 3.26: Containerized Deployment Architecture

3.6 Future Work

This section proposes enhancements to the Restaurant Management Software to further improve its functionality and impact, focusing on practical modifications rather than major overhauls like mobile versions.

Future developments include integrating a predictive analytics module to forecast inventory needs based on historical sales and seasonality, reducing overstocking and waste. Another enhancement is the addition of a customizable alert system, allowing restaurant managers to set personalized thresholds for low stock or expiration dates, improving operational flexibility. Additionally, introducing a multi-language interface (beyond French) to support Arabic and English natively will broaden accessibility for diverse staff. To address the performance limitations of slow end-to-end tests, we plan to implement component tests to validate individual modules more efficiently. Furthermore, recipe creation will be enhanced to allow recipes to be based on other recipes, enabling the reuse of existing ingredients and preparation steps to streamline menu development. These small yet impactful changes aim to enhance decision-making, user experience, and market reach, positioning the software as a continuously evolving solution for restaurant management. Figure 3.27 presents a conceptual overview of these proposed features.

Figure 3.27: Conceptual Overview of Future Features

3.7 Conclusion

This chapter detailed the realization and implementation phase of the Restaurant Management Software, consolidating the development of all modules—Inventory Management, Recipe Management, Menu Management, Order Management, Kitchen Interface, and Reporting—into a functional system. We successfully transformed the designs from Chapter 2 into a cohesive application that addresses the food wastage problem identified in Chapter 1.

The comprehensive testing and validation process verified that the system aligns with Chapter 2's plans, providing effective tools to improve inventory and operational management while reducing food waste. The software is now ready for deployment in real restaurants, where it can enhance efficiency and promote sustainability.

Conclusion and Future Works

The development of the Restaurant Management System for coffee shops and restaurants has successfully addressed the critical operational challenges identified at the project's outset, particularly the issue of food wastage through improved inventory management. This project has demonstrated how a thoughtfully designed digital solution can provide tangible benefits for small to medium-sized businesses in the hospitality industry.

The system's integrated approach, linking inventory directly to recipes, menu items, and orders, provides unprecedented visibility into ingredient usage and enables proactive management of stock levels. This direct connection between what is purchased, what is used, and what is sold represents a significant advancement over traditional systems that treat these areas as separate domains. By automatically tracking inventory depletion based on actual sales, the system eliminates the manual reconciliation processes that are often error-prone and time-consuming.

The implementation of features specifically targeting food wastage reduction—such as expiration date tracking, low stock alerts, and waste logging—directly addresses one of the most significant sources of financial loss in restaurant operations. Even a modest reduction in waste can translate to substantial savings over time, improving both profitability and environmental sustainability.

The adoption of the Kanban methodology, implemented through Trello, proved to be an effective approach for managing the development process. The visual nature of Kanban boards facilitated clear communication about progress and priorities, while the focus on limiting work in progress helped maintain steady momentum throughout the project. This methodology allowed for continuous adaptation based on emerging insights and feedback, which was particularly valuable given the evolving understanding of user needs and technical constraints.

The technical choices made during the project, particularly the selection of Nuxt.js for the frontend and Supabase for the backend, provided a solid foundation for building a responsive, cross-platform application with robust data management capabilities. The implementation of offline functionality addresses a critical need in environments with potentially unreliable internet connectivity, ensuring that essential business operations can continue uninterrupted.

While the current implementation successfully delivers the core functionality required to address the identified problems, several opportunities for future enhancement have been identified:

- **Advanced Analytics:** Implementing more sophisticated predictive analytics for inventory needs based on historical sales patterns, seasonal trends, and external factors.
- **Supplier Management:** Adding functionality for managing suppliers, automating purchase orders based on inventory levels, and tracking order status.

- **Enhanced Mobile Experience:** Developing native mobile applications for improved performance and device integration on smartphones.
- **Integration Capabilities:** Creating interfaces with accounting software, e-commerce platforms, and supplier systems for more comprehensive business management.

These potential enhancements would build upon the solid foundation established by the current system, further extending its capabilities and value to restaurant operators.

In conclusion, the Restaurant Management System developed through this project represents a significant advancement in addressing the operational challenges faced by coffee shops and restaurants. By providing tools that directly target food wastage and improve operational efficiency, the system offers tangible benefits that can positively impact both profitability and sustainability. The successful implementation of this project demonstrates the value of applying modern web technologies and thoughtful design principles to solve real-world business problems.

Bibliography

[Con21] Database Systems: A Practical Approach to Design, Implementation, and Management, Connolly, Thomas and Begg, Carolyn, Pearson, London, UK, 2021 (6th ed.)

[And22] Kanban: Successful Evolutionary Change for Your Technology Business, Anderson, David J., Blue Hole Press, Seattle, WA, 2022 (2nd ed.)

[Wil23] Effective Team Dynamics in Agile Projects, Wilson, Claire, Journal of Team Management, 18(4), 55–70, 2023, Management Press

[Car23] Designing User-Friendly Interfaces for Business Applications, Carter, Anna, Journal of User Experience, 17(1), 20–35, 2023, UX Press

[Hal22] Standards for Web Application Development in 2022, Hall, Richard, Journal of Web Technologies, 8(3), 70–85, 2022, Web Tech Press

[Zha24] Operational Challenges in Small-Scale Restaurants, Zhang, Li, Academic Press, New York, NY, 2024

[Smi22] Trends in Restaurant Technology: A 2022 Review, Smith, John, Journal of Hospitality Management, 45(3), 123–135, 2022, Hospitality Press

Webography

[PWright25] Playwright, *e2e testing with Playwright*, consulted on May 28, 2025, https://playwright.dev/docs/library

[Tun23] Tech Tunisia Network, *Market Trends in Tunisian Tech*, consulted on May 22, 2025, https://www.techtunisia.org/trends

[Cursor25] WebTech Solutions, *Best code editor in 2025*, consulted on May 22, 2025, https://docs.cursor.com/faq

[Git25] WebTech Solutions, *github as a source control tool*, consulted on May 29, 2025, https://docs.github.com/en/get-started/using-github/github-flow

[Nuxt25] WebTech Solutions, *nuxtjs as frontend, based on vuejs*, consulted on May 29, 2025, https://nuxt.com/docs

[SupBas25] WebTech Solutions, *supabase the opensouce firebase alternative*, consulted on May 29, 2025, https://supabase.com/docs

[NRD22] National Resources Defense Council, *The Financial Impact of Food Waste*, consulted on May 22, 2025, https://www.nrdc.org/food-waste-impact

[FAO23] Food and Agriculture Organization, *Global Food Loss and Waste*, consulted on May 22, 2025, https://www.fao.org/food-loss-and-waste

[Hos22] Hospitality Tech, *Technology Trends in Hospitality*, consulted on May 23, 2025, https://www.hospitalitytech.com/trends

[R36523] Restaurant365 Inc., *Product Overview*, consulted on May 24, 2025, https://www.restaurant365.com/overview

[Mar23] MarketMan Inc., *Features and Benefits*, consulted on May 25, 2025, https://www.marketman.com/features

[Pla23] Plate IQ Inc., *Solution Details*, consulted on May 22, 2024, https://www.plateiq.com/details

[NRA24] National Restaurant Association, *2024 Restaurant Industry Challenges*, consulted on May 23, 2025, https://www.restaurant.org/challenges-2024

[Atl23] Atlassian, *Trello Guide for Teams*, consulted on May 26, 2025, https://www.atlassian.com/trello-guide

[Tre22b] Trello Blog, *Exploring Trello Features*, consulted on May 26, 2025, https://blog.trello.com/features

[Tech23] TechRadar, *Benefits of Trello*, consulted on May 22, 2025, https://www.techradar.com/trello-benefits https://blog.trello.com/features

[Collab23] Forbes Tech, *Top Collaboration Tools for Teams in 2023*, consulted on June 4, 2025, https://www.forbes.com/tech/collaboration-tools-2023

[BWaste25] businessWaste, *Catastrophic Food Wastage*, consulted on May 31, 2025, https://www.businesswaste.co.uk/sectors/restaurant-waste/restaurant-waste-statistics/

[TheResto25] TheRestaurantHQ, *Catastrophic Financial Loss*, consulted on May 31, 2025, https://www.therestauranthq.com/trends/restaurant-food-waste-statistics/

[ResGate25] ResearchGate, *Bread Wastage in tunisia*, consulted on June 2, 2025, https://www.researchgate.net/publication/311101497_FOOD_WASTAGE_BY_TUNISIAN_HOUSEHOLDS

[Docker25] Docker, *Hosting*, consulted on June 2, 2025, https://docs.docker.com/